

BUILD AN LLM FROM SCRATCH IN RUST

23 Assignments · Deep Theory · Tensor-First · No Libraries

23	Assignments
8	Phases
4	Gaps filled
A02	Tensor (was missing)
18+mo	Timeline

This document is self-contained. Every concept section has enough theory that you do not need to look elsewhere. Every assignment has numbered tasks, Rust starter code, stretch goals, and what you will feel when it works.

COMPLETE LEARNING SEQUENCE

A01	A02	A03	A04	A05	A06	A07	A08	A09	A10	A11	G01
A12	A13	A14	A15	G02	A16	G03	A17	A18	G04	A19	

G01 G02 G03 G04 = Gap assignments added from audit of GPAI + ChatGPT + DeepSeek responses. A02 = Tensor struct, previously missing from all three AI responses.

TABLE OF CONTENTS

Phase 1 — Math Foundations

- A01 Vectors & Dot Products — The Language of Similarity (3–5 days)
- A02 Tensor Struct — Foundation for All 21 Remaining Assignments [★ ADDED] (1 week)
- A03 Matrix Multiply as Coordinate Transformation (3–5 days)
- A04 Chain Rule — Gradient Descent Finds the Golden Ratio (4–6 days)
- A05 Softmax & Cross-Entropy — Shakespeare Bigram Model (3–5 days)

Phase 2 — Autograd Engine

- A06 Scalar Autograd — Backprop Through XOR (1 week)
- A07 Tensor Autograd + Gradient Checker — Fibonacci Net (1–2 weeks)

Phase 3 — Neural Networks

- A08 Perceptron — Classify Temperatures, Fail at XOR (3–4 days)
- A09 MLP — MNIST Digit Recognition at >85% Accuracy (1–2 weeks)
- A10 MLP Language Model — Generate Shakespeare, Hit the Ceiling (1–2 weeks)
- A11 AdamW + Batched Training — Break the SGD Ceiling (4–6 days)

Gap 1 — Architecture (RNN)

- G01 RNN — The Vanishing Gradient Problem in Real Numbers [GAP] (1 week)

Phase 4 — Transformer Architecture

- A12 Attention — Trace 'To Be or Not to Be' Token by Token (1 week)
- A13 Multi-Head Attention + RoPE (1–2 weeks)
- A14 Full Transformer Block — Beat the MLP and RNN (1–2 weeks)

Phase 5 — Tokenizer

- A15 BPE Tokenizer — Train on Shakespeare, Measure Compression (1 week)

Gap 2 — Trie Tokenizer

- G02 Trie Vocabulary — $O(N \cdot L)$ Encoding, 150x Faster [GAP] (4–6 days)

Phase 6 — Systems & Speed

- A16 SIMD GEMM — Make Tensor.matmul 18x Faster (1–2 weeks)

Gap 3 — Parallel Pipeline

- G03 Parallel Data Pipeline — From 43% to 97% Compute Utilization [GAP] (4–6 days)

Phase 7 — Complete LLM

- A17 Tiny GPT — 10M Parameters on TinyStories (4–6 weeks)
- A18 KV Cache + Sampling — From 21 to 208 Tokens/Sec (1 week)

Gap 4 — Quantization

- G04 INT8 Quantization — Tensor Weights 40MB to 10MB [GAP] (1 week)

Phase 8 — Rust Coding Model

- A19 Rust Code Completion — The Final Project (4–8 weeks)

Phase 1 — Math Foundations

A01 Vectors & Dot Products — The Language of Similarity

Duration:
3–5 days

Dataset: 5 word vectors: cat, dog, car, truck, love (d=3 dimensions)

Why: Every Q·K attention score is a dot product. Before writing a single neural network line, you need to feel this operation — what it computes geometrically and why a high value means 'similar'. This is the mathematical atom of the entire Transformer.

THEORY & CONCEPT

WHAT IS A VECTOR?

A vector is an ordered list of numbers. Each number is one measurable feature of the thing being represented.

"cat" = [0.2, 0.9, 0.7] (size=small, living=yes, positive=yes)

"car" = [0.8, 0.0, 0.4] (size=large, living=no, positive=neutral)

In a real LLM: d = 4096 (GPT-3) or 7168 (Llama 3 70B). Same math.

DOT PRODUCT:

$u \cdot v = u_0 \cdot v_0 + u_1 \cdot v_1 + \dots + u_n \cdot v_n = ||u|| \cdot ||v|| \cdot \cos(\theta)$

Geometric meaning: the dot product measures how much two vectors point in the same direction.

$\cos(0^\circ) = 1.0 \rightarrow$ same direction $\rightarrow$ maximum similarity

$\cos(90^\circ) = 0.0 \rightarrow$ perpendicular $\rightarrow$ no similarity

$\cos(180^\circ) = -1.0 \rightarrow$ opposite $\rightarrow$ maximum dissimilarity

COSINE SIMILARITY (removes length effect):

$\text{cosine}(u,v) = u \cdot v / (||u|| \cdot ||v||)$

Range: [-1, +1]. Pure direction, no length bias.

$\text{cosine}(\text{cat}, \text{dog}) \approx 0.946$ (both living, small, positive $\rightarrow$ near-identical)

$\text{cosine}(\text{car}, \text{truck}) \approx 0.999$ (essentially same direction)

$\text{cosine}(\text{cat}, \text{car}) \approx 0.157$ (very different in all dimensions)

VECTOR PROJECTION:

$\text{proj}(a \text{ onto } b) = (a \cdot b / ||b||^2) \cdot b$

Returns: component of a that aligns with b.

Used in attention: selecting how much of each value to extract.

LINK TO TRANSFORMER:

$\text{Attention score}(i,j) = \text{dot}(Q_i, K_j) / \sqrt{d_k}$

High dot product = high attention weight = token i attends to token j.

Every information routing decision in GPT is a dot product.

ASSIGNMENT TASKS

- Task 1: Implement `dot(a,b)`, `norm(v)`, `cosine(a,b)`, `normalize(v)`, `project(a,b)`
- Task 2: Compute the full 5x5 similarity matrix for {cat,dog,car,truck,love}
- Task 3: Verify: `cosine(cat,dog)=0.946`, `cosine(car,truck)=0.999`, `cosine(cat,car)=0.157`
- Task 4: Find highest and lowest off-diagonal pair. Does it match intuition?
- Task 5: Project 'cat' onto 'dog'. Verify `dot(projection, residual) < 1e-5` (perpendicular)

RUST STARTER CODE

```
fn dot(a:&[f32],b:&[f32])->f32{a.iter().zip(b).map(|(x,y)|x*y).sum()} fn norm(v:&[f32])->f32{dot(v,v).sqrt()} fn
cosine(a:&[f32],b:&[f32])->f32{dot(a,b)/(norm(a)*norm(b))} fn normalize(v:&[f32])->Vec<f32>{let
n=norm(v);v.iter().map(|x|x/n).collect()} fn project(a:&[f32],onto:&[f32])->Vec<f32>{ let
s=dot(a,onto)/dot(onto,onto); onto.iter().map(|x|x*s).collect() } fn main(){ let
words=["cat","dog","car","truck","love"]; let vecs:&[f32]=&[0.2,0.9,0.7],[0.3,0.9,0.6],
&[0.8,0.0,0.4],[0.9,0.0,0.3],[0.1,0.3,1.0]]; print!("{:<8}", ""); for w in &words{print!("{:>8}",w);} println();
for(i,&w)in words.iter().enumerate(){ print!("{:<8}",w); for j in 0..5{print!("{:>8.3}",cosine(vecs[i],vecs[j]));}
println(); } let proj=project(vecs[0],vecs[1]); let
resid:Vec<f32>=vecs[0].iter().zip(&proj).map(|(a,p)|a-p).collect(); assert!(dot(&proj,&resid).abs()<1e-5,"not
perpendicular!"); println!("perpendicularity: |dot|={:.2e} OK",dot(&proj,&resid).abs()); }
```

STRETCH GOALS

1. Implement `angular_distance(a,b) = arccos(cosine(a,b))` in degrees
2. 'king - man + woman' analogy: which word is closest?

Use vectors: `king=[0.8,0.7,0.2]`, `man=[0.3,0.1,0.5]`, `woman=[0.2,0.1,0.6]`

3. Verify `||normalize(v)|| == 1.0` for all 5 word vectors

RESOURCES

- * 3Blue1Brown — Essence of Linear Algebra Ep.1-3
- * Linear Algebra Done Right Ch.1
- * Karpathy micrograd — first 20 min

WHAT YOU WILL FEEL WHEN IT WORKS:

Print the matrix. `cat<->dog=0.946` while `cat<->car=0.157`. You did not guess — the arithmetic produced it. This is attention at `d=3` instead of `d=4096`.

A02 Tensor Struct — Foundation for All 21 Remaining Assignments

Duration:
1 week

Dataset: Shape [3,4,5] trace: strides, indexing, reshape, matmul with shape checks

Why: PREVIOUSLY MISSING from the roadmap. Without a Tensor struct you pass loose (m,k,n) size parameters to every function and have zero shape checking. When you reach 4D attention tensors `[batch,heads,seq,head_dim]` that becomes impossible to debug. Build this once, use it for every subsequent assignment. Every line of Rust code from A03 onward calls Tensor methods.

THEORY & CONCEPT

WHAT IS A TENSOR?

A tensor is a multi-dimensional array stored as a FLAT `Vec<f32>`.

0D: scalar 3.14

1D: vector [1.0, 2.0, 3.0] shape=[3]

2D: matrix [[1,2],[3,4],[5,6]] shape=[3,2]

```

3D: [batch, seq, features] shape=[8,512,4096]
4D: [batch, heads, seq, head_dim] shape=[8,32,512,128]

ROW-MAJOR LAYOUT (C order, what Rust uses):
Matrix [3 rows x 4 cols] in memory:
[row0_col0, row0_col1, row0_col2, row0_col3,
 row1_col0, row1_col1, ... ]

element[i,j] = flat_index i*4 + j*1
strides = [4, 1] (elements to skip per dimension step)

For shape=[2,3,4]: strides=[12, 4, 1]
element[i,j,k] = i*12 + j*4 + k*1

Formula: stride[d] = product of all dimensions AFTER d

WHY STRIDES MATTER (cache performance, learned in A16):
CPU cache line = 64 bytes = 16 f32s.

Accessing row i: data[i*N+0..i*N+N] stride=1 -> contiguous -> fast
Accessing col j: data[0*N+j, 1*N+j, ...] stride=N -> cache miss every step!

matmul(i,j,k loop) accesses B[p,j] with stride=N -> slow
matmul(i,k,j loop) accesses B[k,j+0..j+N] -> sequential -> fast

This single loop reorder gives 4x speedup (measured in A16).

```

SHAPE CHECKING:

```

Without: matmul([2,3],[4,5]) silently computes garbage.
With: assert_eq!(k, other.shape[0], "inner dim mismatch: {:?} @ {:?}")

Panics immediately with readable message. Saves hours per bug.

LLM TENSOR SHAPES (reference for all future assignments):
Token embeddings: [seq=512, d_model=4096]
Weight W_q: [d_model=4096, d_k=4096]
Q after projection: [seq=512, d_k=4096]
Attention scores: [heads=32, seq=512, seq=512]
FFN hidden: [seq=512, d_ff=16384]
Full batch: [batch=8, seq=512, d_model=4096]

```

ASSIGNMENT TASKS

- Task 1: `Tensor::new(data,shape), ::zeros(shape), ::randn(shape,scale)` [Box-Muller, no crates]
- Task 2: `row_major_strides(shape), idx(indices), get(indices), set(indices,v)`
- Verify: `shape=[3,4,5]` gives `strides=[20,5,1]`; `t.get([1,2,3])==33.0`
- Task 3: `reshape(new_shape)` with `assert, transpose_2d()`
- Task 4: `elem_add, elem_mul, scale, apply(fn), sum, mean, max, min`
- Task 5: `matmul([M,K],[K,N]->[M,N])` using `i,k,j` loop order. Shape-checked `assert`.
- Verify: `[2,3]@[3,2] = [[58,64],[139,154]]`
- Task 6: `softmax_rows()` — row-wise stable softmax on 2D Tensor
- Task 7: `print_info(label)` — prints shape, strides, min, max, mean

RUST STARTER CODE

```
#[derive(Debug,Clone)] pub struct Tensor{pub data:Vec<f32>,pub shape:Vec<usize>,pub strides:Vec<usize>} impl Tensor{
pub fn new(data:Vec<f32>,shape:Vec<usize>)->Self{ let n:usize=shape.iter().product(); assert_eq!(data.len(),n,"data
len {} != shape {:?}",data.len(),shape,n); let strides=Self::strides(&shape); Tensor{data,shape,strides} }
pub fn zeros(s:Vec<usize>)->Self{Tensor::new(vec![0.;s.iter().product()],s)} pub fn
randn(s:Vec<usize>,scale:f32)->Self{ let n:usize=s.iter().product(); let mut d=Vec::with_capacity(n); let mut
seed=12345u64; while d.len()<n{ seed=seed.wrapping_mul(6364136223846793005).wrapping_add(1); let u1=(seed>>33)as
f32/(1u64<<31)as f32+1e-10; seed=seed.wrapping_mul(6364136223846793005).wrapping_add(1); let u2=(seed>>33)as
f32/(1u64<<31)as f32; d.push((-2.*u1.ln()).sqrt()*(2.*std::f32::consts::PI*u2).cos()*scale); } d.truncate(n);
Tensor::new(d,s) } fn strides(s:&[usize])->Vec<usize>{ let mut st=vec![1;size]; for i
in 0..s.len().saturating_sub(1).rev(){st[i]=st[i+1]*s[i+1];} st } pub fn get(&self,idx:&[usize])->f32{
self.data[idx.iter().zip(&self.strides).map(|(i,s)|i*s).sum:<usize>()] } pub fn matmul(&self,o:&Tensor)->Tensor{
let(m,k,n)=(self.shape[0],self.shape[1],o.shape[1]); assert_eq!(k,o.shape[0],"matmul: {:?} @
{:?}",self.shape,o.shape); let mut out=Tensor::zeros(vec![m,n]); for i in 0..m{for p in 0..k{let av=self.data[i*k+p];
for j in 0..n{out.data[i*n+j]+=av*o.data[p*n+j];}} out } pub fn transpose_2d(&self)->Tensor{
let(m,n)=(self.shape[0],self.shape[1]); let mut o=Tensor::zeros(vec![n,m]); for i in 0..m{for j in
0..n{o.data[j*m+i]=self.data[i*n+j];} o } pub fn elem_add(&self,o:&Tensor)->Tensor{
assert_eq!(self.shape,o.shape,"elem_add: {:?} vs {:?}",self.shape,o.shape);
Tensor::new(self.data.iter().zip(&o.data).map(|(a,b)|a+b).collect(),self.shape.clone()) } pub fn
elem_mul(&self,o:&Tensor)->Tensor{
Tensor::new(self.data.iter().zip(&o.data).map(|(a,b)|a*b).collect(),self.shape.clone()) } pub fn
scale(&self,s:f32)->Tensor{ Tensor::new(self.data.iter().map(|x|x*s).collect(),self.shape.clone()) } ... (truncated,
see full code in digital version)
```

STRETCH GOALS

1. Add broadcast_add: shape [3,1] + [1,4] -> [3,4] without copying data
2. Add matmul_blocked(block:usize): tile into blocks that fit L1 cache
3. Benchmark matmul vs matmul_blocked on [512,512]x[512,512]. Target 2x speedup

RESOURCES

- * NumPy internals — strides documentation (numpy.org)
- * PyTorch internals blog (Edward Yang)
- * Rustonomicon ch.3 — data layout
- * ndarray crate source

WHAT YOU WILL FEEL WHEN IT WORKS:

t.get([1,2,3])=33.0. a.matmul(&wrong_shape) panics: 'matmul: [2,3] @ [2,3]'. The struct catches errors at the source. Every subsequent assignment uses this without thinking about it.

A03 Matrix Multiply as Coordinate Transformation

Duration:
3–5 days

Dataset: Word vectors [5,3] projected through trained W [3,2] vs random W

Why: Matrix multiply is not just arithmetic — it is a learned coordinate transformation. Every Q, K, V projection in attention, every FFN weight, is this operation. Build intuition for what W does to the embedding space before implementing it inside a model.

THEORY & CONCEPT

$Y = X \cdot \text{matmul}(W) [N, d_{in}] @ [d_{in}, d_{out}] \rightarrow [N, d_{out}]$

W is a change-of-basis matrix. It measures each input vector's component along d_{out} learned directions.

$Y[i,j] = \text{dot}(\text{row}_i \text{ of } X, \text{col}_j \text{ of } W)$

```

= "how much does input i point in direction j?"

EXPERIMENT – trained W [3,2] extracts (creature-ness, positivity):
W = [[ 0.8, 0.1], // size dim
      [-0.2, 0.3], // living dim
      [ 0.6, -0.1]] // positive dim

After Y = X.matmul(&W):
cat -> [0.838, 0.754] high creature, high positive
dog -> [0.822, 0.682] high creature, medium positive
car -> [0.880, 0.120] size-creature, low positive
love -> [0.000, 0.930] zero creature, very positive

With RANDOM W: all points scatter – no semantic structure.
Training FINDS the W that makes geometry meaningful.

LOOP ORDER – why i,k,j beats i,j,k:
i,j,k: inner loop B[p,j] for fixed j, varying p -> stride=N -> cache miss
i,k,j: load A[i,k] once, inner loop B[k, j..j+N] -> sequential -> fast

Tensor.matmul must use i,k,j. Verified measurably faster in A16.

```

TRANSFORMER CONNECTION:

```

Q = X.matmul(&W_q) [T,d] @ [d,d_k] -> [T,d_k]
K = X.matmul(&W_k) same
scores = Q.matmul(&K.transpose_2d()) / sqrt(d_k) [T,T]

```

ASSIGNMENT TASKS

- Task 1: Compute $Y = x.matmul(\&w_trained)$. Verify $Y.shape == [5,2]$
- Task 2: Print each word's 2D coordinates. Verify $cat[0] \approx 0.838$, $love[1] \approx 0.930$
- Task 3: Compute $Y2 = x.matmul(\&Tensor::randn([3,2],0.5))$. Print $Y2$.
- Task 4: Compute $\cosine(Y[cat], Y[dog])$ and $\cosine(Y[cat], Y[car])$ for both W s.
- Trained W ratio should be $>3x$ higher than random W ratio.
- Task 5: Time $matmul_ijk$ vs $matmul_ikj$ on $[256,256] \times [256,256]$. Verify $\geq 2x$ speedup.

RUST STARTER CODE

```

use crate::tensor::Tensor; fn main(){ let x=Tensor::new(vec![ 0.2,0.9,0.7, 0.3,0.9,0.6, 0.8,0.0,0.4, 0.9,0.0,0.3,
0.1,0.3,1.0, ],vec![5,3]); let w=Tensor::new(vec![0.8,0.1,-0.2,0.3,0.6,-0.1],vec![3,2]); let y=x.matmul(&w);
y.print_info("Y=X@W_trained"); let words=["cat","dog","car","truck","love"]; println!("Word creature positivity");
for(i,word)in words.iter().enumerate(){ println!("{:<8} {:<.3} {:<.3}",word,y.data[i*2],y.data[i*2+1]); } // Random W
comparison let w_rand=Tensor::randn(vec![3,2],0.5); let y2=x.matmul(&w_rand); println!("\nWith random W (no semantic
structure):"); for(i,word)in words.iter().enumerate(){ println!("{:<8}
{:<.3},{:<.3}]",word,y2.data[i*2],y2.data[i*2+1]); } }

```

STRETCH GOALS

1. Show $(A @ B).transpose_2d() == B.transpose_2d().matmul(\&A.transpose_2d())$

This identity appears in every matmul backward pass.

2. Implement $Tensor.outter(\&b): [n] \times [m] \rightarrow [n,m]$ $out[i,j]=a[i]*b[j]$

Used in LoRA weight updates.

RESOURCES

- * 3Blue1Brown — Linear transformations Ep.4
- * Gilbert Strang — Lecture 1 MIT 18.06
- * The Matrix Calculus You Need for Deep Learning

WHAT YOU WILL FEEL WHEN IT WORKS:

cat=[0.838,0.754], love=[0.000,0.930] with trained W. With random W: indistinguishable scatter. Training is finding W.

A04

Chain Rule — Gradient Descent Finds the Golden Ratio

Duration:
4–6 days

Dataset: Fibonacci pairs: single weight w converges to $\phi=1.6180339$

Why: The chain rule IS backpropagation. Every dL/dW in a 70B LLM is one chain rule application. Make it concrete on Fibonacci using Tensor — watch the optimizer discover a mathematical constant from raw data.

THEORY & CONCEPT

DERIVATIVE — RATE OF CHANGE:

$f'(x) = \lim_{\epsilon \rightarrow 0} \frac{f(x+\epsilon) - f(x)}{\epsilon}$
"If x increases by tiny ϵ , how much does $f(x)$ change?"

KEY DERIVATIVES TO MEMORIZE:

$\frac{d}{dx} (x^n) = n \cdot x^{(n-1)}$
 $\frac{d}{dx} (e^x) = e^x$
 $\frac{d}{dx} (\tanh x) = 1 - \tanh^2(x)$ <- critical: used in RNN, attention
 $\frac{d}{dx} (\text{sigmoid}) = \text{sigmoid} \cdot (1 - \text{sigmoid})$ <- classification output
 $\frac{d}{dx} (\text{relu } x) = 1 \text{ if } x > 0 \text{ else } 0$ <- MLP activation

CHAIN RULE — DIFFERENTIATING COMPOSITIONS:

$y = f(g(x)) \rightarrow \frac{dy}{dx} = f'(g(x)) \cdot g'(x)$

 $L = \text{loss}(\text{output}(\text{hidden}(\text{input})))$
 $\frac{dL}{d\text{input}} = \frac{dL}{d\text{output}} \cdot \frac{d\text{output}}{d\text{hidden}} \cdot \frac{d\text{hidden}}{d\text{input}}$

This is backpropagation. Each term is a local gradient.
PyTorch multiplies them automatically. You do it manually here.

GRADIENT DESCENT:

$w \leftarrow w - \text{lr} \cdot \frac{dL}{dw}$

The gradient points UPHILL. We move downhill (descent).

lr too large: overshoot minimum, loss diverges

lr too small: converge slowly

Typical: $1e-4$ to $3e-4$ for LLMs

THE FIBONACCI EXPERIMENT:

Fibonacci: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55...

$f(n+1)/f(n) \rightarrow \phi = 1.6180339\dots$ (golden ratio) as $n \rightarrow \infty$

Task: train w to minimize $L = \text{mean}((w \cdot x - y)^2)$

where $x=f(n)$, $y=f(n+1)$

Chain rule derivation:

$dL/d(\text{pred}) = 2 \cdot (\text{pred} - y)$ [MSE gradient]

$d(\text{pred})/dw = x$ [derivative of $w \cdot x$ w.r.t. w]

$dL/dw = (2/N) \cdot \sum((w \cdot x_i - y_i) \cdot x_i)$

NUMERICAL VERIFICATION (use on every gradient you derive):

$\text{numerical} = [L(w+h) - L(w-h)] / 2h$ where $h=1e-5$

If $|\text{analytical} - \text{numerical}| / \max(|a|, |n|) < 1e-3$: your gradient is correct

ASSIGNMENT TASKS

- Task 1: Build $xs = \text{Tensor}([f_0..f_{N-1}], \text{shape}=[N-1, 1])$ and $ys = \text{Tensor}([f_1..f_N], \text{shape}=[N-1, 1])$
- Task 2: Forward: $\text{pred} = xs \cdot w$, $\text{loss} = \text{diff} \cdot \text{elem_mul}(\&\text{diff}).\text{mean}()$ using Tensor ops
- Task 3: Backward: $\text{grad} = (2/N) \cdot \text{sum of } (\text{pred} - ys).\text{data}[i] \cdot xs.\text{data}[i]$
- Task 4: Train 500 steps, print every 100: step, w , loss, $|w - \phi|$
- Target: $|w - 1.6180339| < 1e-5$ after 500 steps
- Task 5: Numerical gradient check at step 0. Must match within $1e-4$.

RUST STARTER CODE

```
use crate::tensor::Tensor; fn main(){ let fib:Vec<f32>=vec![1.,1.,2.,3.,5.,8.,13.,21.,34.,55.,89.]; let
n=fib.len()-1; let xs=Tensor::new(fib[..n].to_vec(),vec![n,1]); let ys=Tensor::new(fib[1..].to_vec(),vec![n,1]); let
mut w=1.0f32; let lr=0.0001f32; for step in 0..=500{ let pred=xs.scale(w); let diff=pred.elem_add(&ys.scale(-1.0));
let loss=diff.elem_mul(&diff).mean(); let grad:f32=xs.data.iter().zip(&diff.data)
.map(|(xi,di)|2.0*di*xi).sum::<f32>()/n as f32; if step==0{ let h=1e-5f32; let
loss_fn=|ww:f32|xs.scale(ww).elem_add(&ys.scale(-1.0)).apply(|v|v*v).mean(); let
num=(loss_fn(w+h)-loss_fn(w-h))/(2.*h); println!("%Gradient check: numerical={:.6} analytical={:.6} err={:.2e}",
num,grad,(num-grad).abs()); } w-=lr*grad; if step%100==0{println!("%Step {:>4}: w={:.8} loss={:.6} phi_err={:.2e}",
step,w,loss,(w-1.6180339f32).abs());} } println!("\nFinal w = {:.10} phi = {:.10}",w,1.6180339887f32);
println!("%Fibonacci ratios converging to phi:"); for i in 2..fib.len(){println!("%
f({})/f({})={:.8}",i,i-1,fib[i]/fib[i-1]);} }
```

STRETCH GOALS

1. Add momentum: $v=0.9 \cdot v + \text{grad}$; $w -= \text{lr} \cdot v$. Compare steps to convergence.
2. Plot the loss curve as ASCII bar chart (one bar per 50 steps).
3. Try learning rates 0.001, 0.0001, 0.00001. Find largest that converges.

RESOURCES

- * 3Blue1Brown — What is a derivative (Calculus Ep.2)
- * 3Blue1Brown — Chain rule (Ep.4)
- * Karpathy micrograd full tutorial (2h)

WHAT YOU WILL FEEL WHEN IT WORKS:

step 500: $w=1.61803398$, $\phi=1.61803399$. Error $< 1e-7$. Gradient descent discovered a mathematical constant from raw data.

A05

Softmax & Cross-Entropy — Shakespeare Bigram Model

Duration:
3–5 days

Dataset: 'to be or not to be that is the question' — 13 unique characters

Why: Softmax + cross-entropy is the output and loss of every LLM ever trained. Build it using your Tensor. Train a bigram model and watch perplexity drop from 13 to 2.1 — your first working language model.

THEORY & CONCEPT

SOFTMAX — CONVERTING SCORES TO PROBABILITIES:

```
softmax(x)_i = exp(x_i) / sum_j(exp(x_j))
```

Properties: all outputs in (0,1), sum to 1, largest input -> largest prob.

NUMERICAL STABILITY PROBLEM:

```
exp(1000) = infinity in float32 -> overflow -> NaN
```

FIX — subtract max first (mathematically identical, numerically safe):

```
softmax(x)_i = exp(x_i - max(x)) / sum_j(exp(x_j - max(x)))
```

Always use the stable version. NaN from unstable softmax is a common cause of mysterious training failures.

CROSS-ENTROPY LOSS:

```
L = -log(softmax(x)[target])
```

```
= -(x[target] - log(sum_j exp(x_j)))
```

Numerically stable log-sum-exp form:

```
max_x = max(x)
```

```
lse = log(sum_j exp(x_j - max_x)) + max_x
```

```
L = lse - x[target]
```

Range: L->0 when prob[target]->1 (perfect), L->inf when prob[target]->0

GRADIENT (beautiful result):

```
dL/dx[i] = softmax(x)[i] - 1{i == target}
```

```
= predicted_probability - true_probability
```

For correct class: grad = prob[target] - 1 (negative, pushes up)

For wrong classes: grad = prob[i] (positive, pushes down)

This single formula is used in every LLM training step.

PERPLEXITY — THE LANGUAGE MODEL METRIC:

```
ppl = exp(mean cross-entropy loss)
```

ppl=K means model is as confused as choosing from K equal options.

ppl=13: random (knows nothing about 13-char vocab)

ppl=2: knows next char is likely one of ~2 candidates

GPT-4 English text: ppl~8. Random model: ppl=50000 (vocab size).

ASSIGNMENT TASKS

■ Task 1: softmax_stable(logits:&Tensor)->Tensor (1D, uses x.apply+scale)

- Task 2: `cross_entropy(logits:&Tensor, target:usize)->f32` (stable log-sum-exp)
- Task 3: `ce_grad(logits:&Tensor, target:usize)->Tensor` (softmax minus one-hot)
- Task 4: Numerical gradient check on `logits=[0.5,-0.2,1.1,-0.8]`, `target=2`
- All 4 components within `rel_err < 1e-3`
- Task 5: Train bigram model 500 steps. Target: `ppl < 2.5`
- Task 6: Print learned bigram table. Verify 'b'->'e' at >90% (only option in text)

RUST STARTER CODE

```
use crate::tensor::Tensor; fn softmax_stable(x:&Tensor)->Tensor{ let max=x.max(); let e=x.apply(|v|(v-max).exp());
e.scale(1.0/e.sum()) } fn cross_entropy(x:&Tensor,t:usize)->f32{ let max=x.max(); let
lse=x.data.iter().map(|&v|(v-max).exp()).sum::<f32>().ln()+max; lse-x.data[t] } fn
ce_grad(x:&Tensor,t:usize)->Tensor{ let mut g=softmax_stable(x); g.data[t]=-1.0; g } fn main(){ // Gradient check let
logits=Tensor::new(vec![0.5,-0.2,1.1,-0.8],vec![4]); let anal=ce_grad(&logits,2); let h=1e-5f32; for i in 0..4{ let
mut lp=logits.clone(); lp.data[i]+=h; let mut lm=logits.clone(); lm.data[i]-=h; let
num=(cross_entropy(&lp,2)-cross_entropy(&lm,2))/(2.*h); let err=(num-anal.data[i]).abs(); println!("{}", num={:.5}
anal={:.5} err={:.2e} {}",i,num,anal.data[i],err,if err<1e-3{"OK"}else{"FAIL"}); } // Bigram model let text="to be or
not to be that is the question"; let mut
chars:Vec<char>=text.chars().collect::<std::collections::HashSet<>>().into_iter().collect(); chars.sort(); let
v=chars.len(); let stoi:std::collections::HashMap<char,usize>=chars.iter().enumerate().map(|(i,&c)|(c,i)).collect();
let ids:Vec<usize>=text.chars().map(|c|stoi[&c]).collect(); let mut w=Tensor::zeros(vec![v,v]); for step in 0..=500{
let(mut loss,mu n)=(0.0f32,0usize); for pair in ids.windows(2){ let(inp,tgt)=(pair[0],pair[1]); let
logits=Tensor::new(w.data[inp*v..(inp+1)*v].to_vec(),vec![v]); loss+=cross_entropy(&logits,tgt); let
g=ce_grad(&logits,tgt); for j in 0..v{w.data[inp*v+j]-=0.1*g.data[j];} n+=1; } if step%100==0{println!("step {:>4}:
ppl={:.2}",step,(loss/n as f32).exp());} } }
```

STRETCH GOALS

1. Add temperature: `softmax(logits/T)`. Compare `T=0.5` (peaked), `T=1.0`, `T=2.0`.
2. Add top-k filtering: set all but top-k logits to `-inf` before softmax.
3. Compute entropy $H(c)=-\sum p \cdot \log(p)$ per character. Which has highest uncertainty?

RESOURCES

- * StatQuest — Cross Entropy Loss (12 min)
- * Karpathy makemore Ep.1 — bigram model
- * Deep Learning Book §6.2

WHAT YOU WILL FEEL WHEN IT WORKS:

ppl 13.0 -> 2.1. 'b' predicts 'e' at 90%+. Your first language model, from scratch, using only Tensor ops and 500 gradient steps.

Phase 2 — Autograd Engine

A06 Scalar Autograd — Backprop Through XOR

Duration:
1 week

Dataset: XOR truth table: 4 points, 2-layer net, 9 parameters

Why: Build autograd for scalars first — pure graph structure, no tensor complexity. `Rc<RefCell<>>` in Rust forces you to solve the same ownership problem PyTorch solves with Python's GC. Solve XOR — the canonical proof that backprop works.

THEORY & CONCEPT

COMPUTATION GRAPH:

A DAG (Directed Acyclic Graph) where:

- Nodes are Values (scalars with data + grad)
- Edges represent operations (which inputs produced this output)

During forward pass: compute data values AND record the graph.

During backward pass: traverse in reverse, call `_backward()` on each node.

RUST OWNERSHIP PROBLEM:

`C = A + B`. `C._backward()` must mutate BOTH `A.grad` and `B.grad`.

`A` and `B` may also feed into other nodes `D`, `E`, `F`.

-> Multiple owners needed -> `Rc<RefCell<Value>>`

`Rc<T>`: shared ownership (reference counting)

`RefCell<T>`: interior mutability (runtime borrow checking)

`Rc<RefCell<>>`: multiple owners, each can mutate -> computation graph

NOT a hack. PyTorch uses Python's GC for the same purpose.

This is the correct Rust pattern for ML graphs.

CHAIN RULE IN THE GRAPH:

For `C = A * B`:

forward: `C.data = A.data * B.data`

backward: `A.grad += C.grad * B.data` ($dC/dA = B$)

`B.grad += C.grad * A.data` ($dC/dB = A$)

Each `_backward()` closure captures `A`, `B`, `C` by `Rc::clone`.

It reads `C.grad` (upstream), writes into `A.grad` and `B.grad`.

TOPOLOGICAL SORT:

Must call `_backward()` in reverse topological order.

Otherwise a node's gradient isn't complete when its `_backward` runs.

Algorithm: DFS, append to topo AFTER visiting all children.

Reverse topo = correct backprop order.

XOR TEST:

```
[0,0]->0 [0,1]->1 [1,0]->1 [1,1]->0
Net: [x1,x2] -> Linear(2->2) -> tanh -> Linear(2->1) -> tanh -> pred
9 parameters. Training with lr=0.1.
step 300: predictions ≈ [0.04, 0.96, 0.95, 0.05] <- SOLVED
```

ASSIGNMENT TASKS

- Task 1: Implement `Value::new(f32)->Rc<RefCell<Value>>` with `data,grad,bwd,prev` fields
- Task 2: Implement `add(a,b)`, `mul(a,b)`, `tanh(a)` — each creates output node with `_backward`
- Task 3: Implement `backward(root)` — topological sort + reverse traversal
- Task 4: Build 2-layer XOR net (9 Values), train 300 steps
- Task 5: Print predictions at step 300. Must be within 0.1 of [0,1,1,0]
- Task 6: Numerical gradient check — verify all 9 gradients within `rel_err < 1e-3`

RUST STARTER CODE

```
use std::rc::Rc; use std::cell::RefCell; type V=Rc<RefCell<Val>>; struct Val{data:f32,grad:f32,bwd:Box<dyn
Fn(>,prev:Vec<V>}> impl Val{ fn new(d:f32)->V{Rc::new(RefCell::new(Val{data:d,grad:0.,
bwd:Box::new(||{}),prev:vec![]}))} fn add(a:&V,b:&V)->V{ let out=Val::new(a.borrow().data+b.borrow().data);
let(ac,bc,oc)=(Rc::clone(a),Rc::clone(b),Rc::clone(&out)); out.borrow_mut().bwd=Box::new(move||{ let
g=oc.borrow().grad; ac.borrow_mut().grad+=g; bc.borrow_mut().grad+=g;});
out.borrow_mut().prev=vec![Rc::clone(a),Rc::clone(b)]; out} fn mul(a:&V,b:&V)->V{ let
out=Val::new(a.borrow().data*b.borrow().data); let(ac,bc,oc)=(Rc::clone(a),Rc::clone(b),Rc::clone(&out));
let(ad,bd)=(a.borrow().data,b.borrow().data); out.borrow_mut().bwd=Box::new(move||{ let g=oc.borrow().grad;
ac.borrow_mut().grad+=g*bd; bc.borrow_mut().grad+=g*ad;}); out.borrow_mut().prev=vec![Rc::clone(a),Rc::clone(b)];
out} fn tanh(a:&V)->V{ let t=a.borrow().data.tanh(); let out=Val::new(t); let(ac,oc)=(Rc::clone(a),Rc::clone(&out));
out.borrow_mut().bwd=Box::new(move||{ ac.borrow_mut().grad+=oc.borrow().grad*(1.-t*t);});
out.borrow_mut().prev=vec![Rc::clone(a)]; out} fn backward(root:&V){ let mut topo=vec![]; let mut
seen=std::collections::HashSet::new(); fn visit(n:&V,t:&mut Vec<V>,s:&mut std::collections::HashSet<*const
RefCell<Val>>){ if s.insert(Rc::as_ptr(n)){ n.borrow().prev.iter().for_each(|c|visit(c,t,s));t.push(Rc::clone(n));}}
visit(root,&mut topo,&mut seen); root.borrow_mut().grad=1.; topo.iter().rev().for_each(|n|(n.borrow().bwd)());}}
```

STRETCH GOALS

1. Add `exp()` and `log()` ops. Build `cross_entropy` using them.
Verify gradient matches the direct formula: `softmax - one_hot(target)`.
2. Add `sub(a,b)` and `pow(a,n)`. Build MSE loss using them.
3. Switch `Rc<RefCell<>>` to `Arc<Mutex<>>` and show it compiles.
This is needed for multi-threaded training (A07).

RESOURCES

- * Karpathy micrograd tutorial (2h — watch all)
- * ferric-micrograd crate (Rust port)
- * Rust Book ch.15 — Rc and RefCell

WHAT YOU WILL FEEL WHEN IT WORKS:

step 300: [0.04, 0.96, 0.95, 0.05]. XOR solved. 9 chain rule applications in a 7-line graph. This is the proof backprop works.

A07 Tensor Autograd + Gradient Checker — Fibonacci Net

Duration:
1–2
weeks

Dataset: Fibonacci: $[f(n), f(n+1)] \rightarrow f(n+2)$. Net discovers addition.

Why: Scale scalar autograd to Tensors. The matmul backward pass ($dL/dA = dL/dC @ B^T$) is the hardest gradient to derive and the most important to verify. Build the gradient checker now — you will use it on every op forever.

THEORY & CONCEPT

TENSOR AUTOGRAD — SAME STRUCTURE, TENSOR FIELDS:

```
TensorNode { data:Tensor, grad:Tensor, backward:Box<dyn Fn()+Send>, prev:Vec<...> }
```

```
Switch Rc<RefCell<>> -> Arc<Mutex<>>
```

Reason: training loop will use Rayon for parallel batch processing.

Rc is NOT Send (cannot cross thread boundaries). Arc is.

MATMUL BACKWARD — THE HARD ONE:

```
C = A.matmul(&B) shapes: [M,K] @ [K,N] -> [M,N]
```

Deriving dL/dA :

```
C[i,j] = sum_k A[i,k] * B[k,j]
```

```
dC[i,j]/dA[i,k] = B[k,j]
```

```
dL/dA[i,k] = sum_j dL/dC[i,j] * B[k,j] <- dot product!
```

```
= (dL/dC @ B^T)[i,k]
```

```
=> dL/dA = dL/dC.matmul(&B.transpose_2d()) shape: [M,N]@[N,K]->[M,K] OK
```

Deriving dL/dB :

```
dL/dB[k,j] = sum_i A[i,k] * dL/dC[i,j]
```

```
= (A^T @ dL/dC)[k,j]
```

```
=> dL/dB = A.transpose_2d().matmul(&dL/dC) shape: [K,M]@[M,N]->[K,N] OK
```

Both backward gradients are just more matrix multiplies.

GRADIENT CHECKER — BUILD THIS, USE IT FOREVER:

```
numerical[i] = [loss(w[i]+h) - loss(w[i]-h)] / 2h (h=1e-5)
```

For each parameter i:

perturb $w[i]$ by $+h$, compute $loss_plus$

perturb $w[i]$ by $-h$, compute $loss_minus$

```
numerical = (loss_plus - loss_minus) / (2*h)
```

```
rel_err = |numerical - analytical| / (|numerical| + |analytical| + 1e-8)
```

PASS if $rel_err < 1e-3$

Use FLOAT64 for numerical check even if model uses float32.

float32 has only ~7 significant digits; numerical diff loses precision.

Run this on EVERY new op before moving on. Saves hours per bug.

FIBONACCI 2->1:

Input: $[f(n), f(n+1)] \rightarrow \text{Linear}(2 \rightarrow 4) \rightarrow \text{ReLU} \rightarrow \text{Linear}(4 \rightarrow 1)$

True answer: $f(n+2) = f(n) + f(n+1)$

After training: output weights converge to [1.0, 1.0] (addition!)

ASSIGNMENT TASKS

- Task 1: Build `TensorNode` with `Arc<Mutex<>>` (not `Rc<RefCell<>>`)
- Task 2: Implement `matmul_node(a,b)->TensorNode` with correct backward
- Task 3: Implement `relu_node`, `add_node` using `Tensor.apply` and `elem_add`
- Task 4: Build gradient checker fn `grad_check(loss_fn,params,grads)->bool`
- Task 5: Run gradient checker on `matmul_node [4,3]@[3,5]`. All 32 params must pass.
- Task 6: Train Fibonacci 2->1 net 200 steps. Output weights must be within 0.05 of [1,1].

RUST STARTER CODE

```
use std::sync::{Arc,Mutex}; use crate::tensor::Tensor; pub fn
grad_check<F:Fn(&[f32])->f32>(f:F,p:&[f32],g:&[f32])->bool{ let h=1e-5f32; let mut ok=true; for i in 0..p.len(){ let
mut pp=p.to_vec(); pp[i]+=h; let mut pm=p.to_vec(); pm[i]-=h; let num=(f(&pp)-f(&pm))/(2.*h); let
rel=(num-g[i]).abs()/(num.abs()+g[i].abs()+1e-8); let pass=rel<1e-3; if !pass{ok=false;} println!("p[{:>2}]
num={:>10.6} anal={:>10.6} rel={:.1e} {}", i,num,g[i],rel,if pass{"OK"}else{"FAIL"}); } ok } // matmul backward: //
dL/dA = dL/dC @ B^T // dL/dB = A^T @ dL/dC pub fn
matmul_node(a:&Arc<Mutex<TNode>>,b:&Arc<Mutex<TNode>>)->Arc<Mutex<TNode>>{ let c_data={let al=a.lock().unwrap();let
bl=b.lock().unwrap(); al.data.matmul(&bl.data)}; let c_shape=c_data.shape.clone(); let c=Arc::new(Mutex::new(TNode{
data:c_data, grad:Tensor::zeros(c_shape), bwd:Box::new(||{}), prev:vec![Arc::clone(a),Arc::clone(b)]}));
let(ac,bc,cc)=(Arc::clone(a),Arc::clone(b),Arc::clone(&c)); c.lock().unwrap().bwd=Box::new(move||{ let
cg=cc.lock().unwrap().grad.clone(); let ad=ac.lock().unwrap().data.clone(); let bd=bc.lock().unwrap().data.clone();
let da=cg.matmul(&bd.transpose_2d()); let db=ad.transpose_2d().matmul(&cg); let mut al=ac.lock().unwrap();
al.grad.data.iter_mut().zip(&da.data).for_each(|(g,d)|*g+=d); let mut bl=bc.lock().unwrap();
bl.grad.data.iter_mut().zip(&db.data).for_each(|(g,d)|*g+=d); }); c }
```

STRETCH GOALS

1. Run gradient checker on the full 2-layer Fibonacci net (all 13 params).
2. Add `LayerNorm` backward. Verify with gradient checker.
3. Profile: how much slower is `Arc<Mutex<>>` vs `Rc<RefCell<>>` on a 100-node graph?

RESOURCES

- * The Matrix Calculus You Need for Deep Learning
- * Rust Book ch.16 — Arc and Mutex
- * Karpathy backprop ninja (makemore pt.4)

WHAT YOU WILL FEEL WHEN IT WORKS:

All parameters print OK. Output weights [1.02, 0.98]. The net found addition from Fibonacci data. Gradient checker never lies.

Phase 3 — Neural Networks

A08 Perceptron — Classify Temperatures, Fail at XOR

Duration:
3–4 days

Dataset: 30 days of temperatures (Celsius) -> hot/cold binary label

Why: The perceptron is the simplest possible learning machine. Seeing it master the 25C threshold — then completely fail at XOR — makes hidden layers feel necessary rather than arbitrary. This is the historical arc of AI from 1958 to 1986.

THEORY & CONCEPT

PERCEPTRON (Frank Rosenblatt, 1958):

$\text{pred} = \text{sigmoid}(w * x + b)$

$L = -y * \log(\text{pred}) - (1-y) * \log(1-\text{pred})$ (binary cross-entropy)

$dL/dw = (\text{pred} - y) * x$

$dL/db = \text{pred} - y$

Can only learn a LINEAR decision boundary.

SIGMOID:

$\text{sigmoid}(x) = 1 / (1 + \exp(-x))$

Range: (0,1). Maps any real to a probability.

$d/dx \text{sigmoid}(x) = \text{sigmoid}(x) * (1 - \text{sigmoid}(x))$

DECISION BOUNDARY:

$\text{pred} = 0.5$ when $\text{sigmoid}(w*x+b) = 0.5$

$\Rightarrow w*x + b = 0$

$\Rightarrow x = -b/w$ (in 1D)

After training on temperature data:

$x_{\text{boundary}} = -b/w \rightarrow \text{denormalize} \rightarrow \text{temperature in Celsius}$

Should converge to ~25C (our threshold).

WHY PERCEPTRON FAILS AT XOR:

XOR: [0,0]->0, [0,1]->1, [1,0]->1, [1,1]->0

Plot these 4 points. They are NOT linearly separable.

No single straight line divides 0s from 1s.

Perceptron can only draw one straight line. Stuck at loss=0.25.

HISTORICAL NOTE:

Minsky & Papert (1969) proved this limitation in "Perceptrons".

Their book nearly ended AI research.

The fix (hidden layers + backprop) wasn't proven until 1986 (Rumelhart et al).

You reproduced the 1969 result in A08 and the 1986 fix in A06.

ASSIGNMENT TASKS

- Task 1: Implement $\text{sigmoid}(x:f32) \rightarrow f32$ and its derivative
- Task 2: Normalize temperatures to $[0,1]$: $x_norm = (\text{temp} - 14) / (34 - 14)$
- Task 3: Train perceptron 300 steps on temperature data. Print loss and boundary every 100 steps.
- Task 4: Verify boundary converges to -25.0 ± 1.0 Celsius
- Task 5: Try XOR with same perceptron 1000 steps. Show it cannot converge (loss stays ~ 0.25).
- Print "PERCEPTRON CANNOT SOLVE XOR — need hidden layers"

RUST STARTER CODE

```
use crate::tensor::Tensor; fn sigmoid(x:f32)->f32{1./(1.+(-x).exp())} fn main(){ let
temps:Vec<f32>=vec![14.,18.,22.,26.,31.,28.,19.,24.,27.,33., 16.,21.,29.,25.,30.,17.,23.,26.,28.,32.]; let
labels:Vec<f32>=vec![0.,0.,0.,1.,1.,1.,0.,0.,1.,1., 0.,0.,1.,0.,1.,0.,0.,1.,1.,1.]; let(tmin,tmax)=(14.0f32,34.0f32);
let x=Tensor::new(temps.iter().map(|t|(t-tmin)/(tmax-tmin)).collect(),vec![20]); let(mut w,mut b)=(0.0f32,0.0f32);
for step in 0..=300{ let(mut dw,mut db,mut loss)=(0.,0.,0.0f32); for(&xi,&yi)in x.data.iter().zip(labels.iter()){ let
p=sigmoid(w*xi+b); loss+=-yi*p.ln()-(1.-yi)*(1.-p).ln(); dw+=(p-yi)*xi; db+=p-yi; } w-=0.1*dw/20.; b-=0.1*db/20.; if
step%100==0{ let boundary=(-b/w)*(tmax-tmin)+tmin; println!("step {:>3}: loss={:.4}
boundary={:.1}C",step,loss/20.,boundary); } } // XOR failure let
xor=[([0.,0.],[0.]),([0.,1.],[1.]),([1.,0.],[1.]),([1.,1.],[0.])]; let(mut w1,mut w2,mut bx)=(0.5f32,-0.5,0.1); for _ in
0..1000{for&([a,b_],y)in&xor{ let p=sigmoid(w1*a+w2*b_+bx); let e=p-y; w1-=0.1*e*a; w2-=0.1*e*b_; bx-=0.1*e; }}
println!("\nXOR after 1000 steps (should be 0,1,1,0:"); for&([a,b_],_)in&xor{println!(" [{a},{b_}] ->
{:.3}",sigmoid(w1*a+w2*b_+bx)); } println!("All near 0.5 -> PERCEPTRON CANNOT SOLVE XOR"); }
```

STRETCH GOALS

1. Plot sigmoid curve as ASCII from $x=-5$ to $x=5$.
2. Implement binary accuracy metric. Show temperature classifier reaches 95%+.
3. Use Tensor for the temperature data and label tensors throughout.

RESOURCES

- * 3Blue1Brown — But what is a neural network Ep.1
- * StatQuest — Perceptron
- * Deep Learning Book §6.1

WHAT YOU WILL FEEL WHEN IT WORKS:

boundary=25.1C. XOR: all near 0.5, stuck. That failure is the 1969 crisis that nearly killed AI research. You reproduced it in 30 lines.

A09 MLP — MNIST Digit Recognition at >85% Accuracy

Dataset: MNIST: 784-pixel images, 10 digit classes, 500 train / 100 test

Duration:
1–2
weeks

Why: MNIST is the canonical proof that your implementation is correct. 85% accuracy is a binary pass/fail. Either your Tensor matmuls, gradients, and training loop all work correctly — or they don't. There is no partial credit.

THEORY & CONCEPT

MULTI-LAYER PERCEPTRON (MLP):

Input(784) -> Linear -> ReLU -> Linear -> ReLU -> Linear -> Softmax

The hidden layers learn non-linear features.

Layer 1 might learn: "is there a vertical stroke?"

Layer 2 might learn: "does it have an enclosed region?"

Layer 3 combines features: "enclosed region + curve = 8 or 9?"

RELU — WHY NOT SIGMOID:

sigmoid gradient: $s \cdot (1-s)$ max value = 0.25 (at $s=0.5$)

After 3 layers: $\text{grad} \approx 0.25 \cdot 0.25 \cdot 0.25 = 0.016$ (64x smaller)

After 10 layers: $\text{grad} \sim 0$ (vanished) <- the vanishing gradient problem

ReLU gradient: 1 if $x > 0$ else 0 <- no shrinkage for positive activations

ReLU doesn't saturate in the positive region.

This is why deep networks use ReLU, not sigmoid.

Dying ReLU: if all inputs to a neuron are negative, it never fires.

Fix: small constant lr, good init, BatchNorm (not implemented here).

INITIALIZATION — XAVIER:

For ReLU: $\text{std} = \sqrt{2/\text{fan_in}}$ (He initialization)

For tanh: $\text{std} = \sqrt{1/\text{fan_in}}$ (Xavier initialization)

Too large init: activations explode through layers

Too small init: activations vanish through layers

Correct init: $\| \text{activations} \|$ stays ~constant across layers

MNIST BINARY FORMAT:

Images file: [magic=2051][N count][rows=28][cols=28][N*784 bytes]

Labels file: [magic=2049][N count][N label bytes]

Parse with `from_be_bytes()` - big-endian format!

Normalize pixel values: $\text{byte}/255.0 \rightarrow [0.0, 1.0]$

ARCHITECTURE: `Linear(784->128)+ReLU + Linear(128->64)+ReLU + Linear(64->10)`

Total params: $784 \cdot 128 + 128 \cdot 64 + 64 \cdot 10 + 10 = 109,386$

ASSIGNMENT TASKS

- Task 1: Parse MNIST binary format. Load 500 train images as `Vec<Tensor>` shape [784].
- Task 2: Implement forward pass using `Tensor.matmul` and `.apply(relu)`
- Task 3: Implement backward pass. Use gradient checker on a mini-net before full training.
- Task 4: Train 50 epochs on 500 samples. Print accuracy every 10 epochs.
- Task 5: Evaluate on 100 test samples. Must achieve >85% test accuracy.
- Task 6: Print confusion matrix [10x10]. Identify top 3 most-confused pairs.

RUST STARTER CODE

```
use crate::tensor::Tensor; fn relu(x:f32)->f32{x.max(0.)} fn relu_grad(x:f32)->f32{if x>0.{1.}else{0.}} fn
parse_images(path:&str)->Vec<Tensor>{ let b=std::fs::read(path).expect("Download MNIST from
yann.lecun.com/exdb/mnist/"); let n=u32::from_be_bytes(b[4..8].try_into().unwrap())as usize;
(0..n).map(|i|Tensor::new( b[16+i*784..16+(i+1)*784].iter().map(|&p|p as f32/255.).collect(), vec![784])).collect() }
fn parse_labels(path:&str)->Vec<usize>{ let b=std::fs::read(path).unwrap(); b[8..].iter().map(|&l|l as
usize).collect() } struct MLP{w1:Tensor,b1:Tensor,w2:Tensor,b2:Tensor,w3:Tensor,b3:Tensor} impl MLP{ fn
new()->Self{let s=0.01f32;MLP{ w1:Tensor::randn(vec![128,784],s),b1:Tensor::zeros(vec![128]),
w2:Tensor::randn(vec![64,128],s), b2:Tensor::zeros(vec![64]), w3:Tensor::randn(vec![10,64],s),
b3:Tensor::zeros(vec![10])}} } fn forward(&self,x:&Tensor)->(Tensor,Tensor,Tensor,Tensor){ let
h1r=self.w1.matmul(&x.reshape(vec![784,1])).reshape(vec![128]).elem_add(&self.b1); let a1=h1r.apply(relu); let
h2r=self.w2.matmul(&a1.reshape(vec![128,1])).reshape(vec![64]).elem_add(&self.b2); let a2=h2r.apply(relu); let
lo=self.w3.matmul(&a2.reshape(vec![64,1])).reshape(vec![10]).elem_add(&self.b3); (lo,h1r,a1,a2) } fn
predict(&self,x:&Tensor)->usize{ let(lo,..)=self.forward(x);
lo.data.iter().enumerate().max_by(|&a,&b|a.1.partial_cmp(b.1).unwrap()).unwrap().0 } }
```

STRETCH GOALS

1. Print one misclassified example as 8x8 ASCII (downsample from 28x28).
2. Try different architectures: [784->256->128->10] vs [784->64->10]. Compare accuracy.
3. Add dropout (randomly zero 20% of activations during training). Does accuracy improve?

RESOURCES

- * MNIST — yann.lecun.com/exdb/mnist/
- * 3Blue1Brown — Gradient descent Ep.2
- * MNIST_In_Rust GitHub reference

WHAT YOU WILL FEEL WHEN IT WORKS:

Epoch 50: 87%. 109,386 gradients computed correctly. The MNIST test is pass/fail — either backprop works or it doesn't. You passed.

A10 MLP Language Model — Generate Shakespeare, Hit the Ceiling

Duration:
1–2
weeks

Dataset: tiny_shakespeare.txt (1.1MB) — context=8 chars, predict char 9

Why: Train until perplexity stops improving. The plateau at ppl~6.2 is not a hyperparameter problem — it is an architectural ceiling. Feeling that ceiling is the motivation for RNN (G01) and then Transformer (A12-A14).

THEORY & CONCEPT

CHARACTER-LEVEL LANGUAGE MODEL:

Context window: 8 characters -> predict character 9

Input: 8 * 65 = 520 one-hot features (65 = Shakespeare vocab size)

Architecture: Linear(520->256)->ReLU->Linear(256->128)->ReLU->Linear(128->65)

Training objective: maximize probability of correct next character

= minimize cross-entropy loss = minimize -log(softmax[correct_char])

WHY THE MLP PLATEAUS:

Fixed weights for all 8 positions.

Position 1 and position 8 use the SAME weight matrix.

The model cannot learn "position 8 is always more relevant than position 1."

It cannot selectively attend. That is what attention does.

More specifically: "ROMEO" and "Juliet" 60 chars apart.

MLP sees context window of 8. ROMEO is never in the same window as Juliet.

The model literally cannot represent this long-range dependency.

GENERATION BY SAMPLING:

At each step: forward pass gives logits for each of 65 chars.

Apply softmax + temperature scaling.

Sample from distribution -> next char.

Append to context, slide window, repeat.

Temperature $T=0.8$ gives good balance of coherence and variety.

$T<0.5$: repetitive, $T>1.5$: gibberish.

PERPLEXITY PROGRESSION:

step 1K: ppl=22 "heeehe eee hhe the the" (learned 'e' is frequent)

step 5K: ppl=9.4 "the king and the lord" (common words emerging)

step 10K: ppl=7.8 "What is the king to speak"

step 50K: ppl=6.2 <- PLATEAU. No further improvement.

step 200K: ppl=6.2 <- Still stuck.

Dataset: raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt

ASSIGNMENT TASKS

- Task 1: Load `tiny_shakespeare.txt`, build char->id mapping, encode as `Vec<usize>`
- Task 2: `context_input(ctx:&[u8], vocab:usize)->Tensor` (concatenated one-hot, shape=[520])
- Task 3: Train MLP 50K steps. Log ppl every 5K steps.
- Task 4: Implement generation: given 8-char seed, generate 200 chars using temperature sampling
- Task 5: Show that training beyond 50K steps does NOT improve ppl significantly
- Task 6: Try context=4 and context=16. Show ppl vs context size table.

RUST STARTER CODE

```
use crate::tensor::Tensor; fn context_input(ctx:&[u8],v:usize)->Tensor{ let mut d=vec![0.0f32;ctx.len()*v];
for(i,&c)in ctx.iter().enumerate(){d[i*v+c as usize]=1.0;} Tensor::new(d,vec![ctx.len()*v]) } fn
sample_token(logits:&Tensor,temp:f32)->usize{ let s=logits.scale(1.0/temp); let max=s.max(); let
e=s.apply(|v|(v-max).exp()); let sum=e.sum(); let mut seed=42u64;
seed=seed.wrapping_mul(6364136223846793005).wrapping_add(1442695040888963407); let r=(seed>>33)as f32/(1u64<<31)as
f32; let mut cum=0.0f32; for(i,&p)in e.data.iter().enumerate(){ cum+=p/sum; if r<cum{return i;} } e.data.len()-1 } //
plateau: ppl=6.2 at 50K regardless of further training // add RNN (G01) and Transformer (A12-A14) to break through
```

STRETCH GOALS

1. Try wider nets: [520->512->256->65]. Does it improve the plateau?
2. Try deeper nets: [520->128->128->128->65]. Same plateau?
3. Plot ppl vs training step as ASCII chart. Show the flattening curve.

RESOURCES

* Karpathy makemore pt.2 — MLP LM

* `tiny_shakespeare.txt` ([raw.githubusercontent.com](https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt))

WHAT YOU WILL FEEL WHEN IT WORKS:

50,000 steps. ppl=6.2. 200,000 steps. ppl=6.2. Architecture is the ceiling. This frustration is what motivated the Transformer.

A11 AdamW + Batched Training — Break the SGD Ceiling

Duration:
4–6 days

Dataset: tiny_shakespeare.txt — batch_size=32, AdamW vs SGD comparison

Why: SGD plateau=6.2, AdamW plateau=5.6. Same model, same data. 9% improvement from changing the optimizer. This proves the optimizer is not a configuration detail — it is half the result.

THEORY & CONCEPT

WHY SGD FAILS ON DEEP NETS:

Each parameter has a very different gradient magnitude.

Embedding gradients: sparse (only updated for tokens that appear)

FFN weight gradients: dense (updated every step)

A global learning rate is simultaneously too large for some params (causes oscillation) and too small for others (too slow).

We need per-parameter adaptive learning rates.

ADAM — ADAPTIVE MOMENT ESTIMATION (Kingma & Ba, 2014):

$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 1st moment (mean of gradients)

$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 2nd moment (variance)

$\hat{m}_t = m_t / (1 - \beta_1^t)$ bias correction

$\hat{v}_t = v_t / (1 - \beta_2^t)$

$w_t = w_{t-1} - lr * \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$

The $\sqrt{\hat{v}_t}$ denominator = estimated gradient std dev.

If a param has high gradient variance: $\hat{v}_t$ large -> smaller effective step.

If a param has consistent gradient: $\hat{v}_t$ small -> larger effective step.

AUTOMATICALLY per-parameter without tuning.

ADAMW — DECOUPLED WEIGHT DECAY (Loshchilov & Hutter, 2017):

Standard Adam: L2 regularization through gradient = wrong (scaled by $1/\sqrt{v}$)

AdamW: apply weight decay directly to weights, independent of gradient:

$w_t = w_{t-1} * (1 - lr * \lambda) - lr * \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$

$\lambda = 0.01$ to 0.1 is typical. Prevents weight explosion.

All production LLMs use AdamW, not Adam.

BIAS CORRECTION:

At $t=1$: $\hat{m}_1 = g_1$ (correct)

Without correction: $m_1 = (1 - 0.9) * g_1 = 0.1 * g_1$ (10x too small at start)

The $(1 - \beta_1^t)$ term corrects this cold-start bias.

GRADIENT CLIPPING:

```
global_norm = sqrt(sum of all grad^2)

if global_norm > max_norm=1.0: scale all grads by max_norm/global_norm

Prevents large gradients in early training from corrupting weights.

Without clipping: NaN loss at step ~200 on Shakespeare.

HYPERPARAMETERS (use these for all LLM assignments):

beta1=0.9, beta2=0.999, eps=1e-8, weight_decay=0.01, max_norm=1.0
```

ASSIGNMENT TASKS

- Task 1: Implement AdamW struct with m,v moment Vecs and step counter
- Task 2: clip_and_update(params:&mut[Tensor], grads:&[Tensor]) with grad clipping
- Task 3: Train Shakespeare MLP with SGD+batch=32 for 20K steps. Record ppl.
- Task 4: Train SAME net with AdamW+batch=32 for 20K steps. Record ppl.
- Task 5: Print comparison table: step | SGD ppl | AdamW ppl
- Task 6: Add cosine LR schedule: warmup 200 steps then cosine decay to lr/10.

RUST STARTER CODE

```
use crate::tensor::Tensor; struct AdamW{lr:f32,b1:f32,b2:f32,eps:f32,wd:f32,step:u64, m:Vec<Tensor>,v:Vec<Tensor>}
impl AdamW{ fn new(shapes:&[Vec<usize>])->Self{ AdamW{lr:3e-4,b1:0.9,b2:0.999,eps:1e-8,wd:0.01,step:0,
m:shapes.iter().map(|s|Tensor::zeros(s.clone())).collect(),
v:shapes.iter().map(|s|Tensor::zeros(s.clone())).collect()} } fn update(&mut
self,params:&mut[Tensor],grads:&[Tensor]){ self.step+=1; let t=self.step as i32;
let(bc1,bc2)=(1.-self.b1.powi(t),1.-self.b2.powi(t)); // Gradient clipping let
norm:f32=grads.iter().flat_map(|g|g.data.iter()).map(|&v|v*v).sum:<f32>().sqrt(); let clip=if
norm>1.{1./norm}else{1.}; for((p,g),m),v)in
params.iter_mut().zip(grads).zip(self.m.iter_mut()).zip(self.v.iter_mut()){ for i in 0..p.data.len(){ let
gi=g.data[i]*clip; m.data[i]=self.b1*m.data[i]+(1.-self.b1)*gi; v.data[i]=self.b2*v.data[i]+(1.-self.b2)*gi*gi; let
mh=m.data[i]/bc1; let vh=v.data[i]/bc2; p.data[i]=p.data[i]*(1.-self.lr*self.wd) -self.lr*mh/(vh.sqrt()+self.eps); }
} } } fn cosine_lr(step:u64,warmup:u64,total:u64,max:f32,min:f32)->f32{ if step<warmup{return max*step as f32/warmup
as f32;} let p=(step-warmup)as f32/(total-warmup)as f32; min+0.5*(max-min)*(1.+(std::f32::consts::PI*p).cos()) }
```

STRETCH GOALS

1. Log gradient norm before and after clipping every 200 steps.
Show that early norms are 8-15x, later norms 0.3-0.8x. Clipping events become rare.
2. Compare beta2=0.999 vs beta2=0.95 (LLM-preferred) on convergence speed.
3. Implement gradient accumulation: accumulate 8 micro-batches before update.
Verify loss is identical to batch_size=32*8=256 (up to rounding).

RESOURCES

- * AdamW paper — Loshchilov and Hutter 2017
- * Deep Learning Book §8 — Optimization
- * Karpathy makemore pt.3

WHAT YOU WILL FEEL WHEN IT WORKS:

SGD: ppl=6.2. AdamW: ppl=5.6. Same model, same data, same architecture. The optimizer changed the result by 9%.

Gap 1 — Architecture (RNN)

G01 RNN — The Vanishing Gradient Problem in Real Numbers

Duration:
1 week

Dataset: tiny_shakespeare.txt — context=40 chars, hidden=128, BPTT

Why: ChatGPT's recommended path: Perceptron->RNN->Transformer. We had Perceptron->MLP->Transformer. The RNN failure is the Transformer's SPECIFIC motivation. Without hitting the vanishing gradient wall in numbers you can print, attention seems arbitrary rather than necessary.

THEORY & CONCEPT

RNN — RECURRENT NEURAL NETWORK:

$$h_t = \tanh(W_{hh} * h_{t-1} + W_{xh} * x_t + b_h)$$

$$y_t = W_{hy} * h_t + b_y$$

The same W_{hh} is applied at EVERY timestep.

h_t compresses ALL previous context into one vector.

In theory: infinite memory. In practice: finite useful range.

BACKPROPAGATION THROUGH TIME (BPTT):

Unroll the RNN T steps, apply standard backprop.

$$dL/dh_0 = dL/dh_T * \text{PRODUCT}(t=1..T) dh_t/dh_{t-1}$$

$$\text{Each factor: } dh_t/dh_{t-1} = W_{hh}^T * \text{diag}(1 - \tanh^2(h_{t-1}))$$

$$\tanh'(x) = 1 - \tanh^2(x)$$

Max value of $\tanh'(x) = 1.0$ (at $x=0$)

But in practice: $\tanh$ saturates, $\tanh'(x) \ll 1$ for $|x| > 1$

THE VANISHING GRADIENT — MEASURED:

With $||W_{hh}|| \sim 0.9$ and typical $\tanh$ saturation:

$$t=39: |dh| = 0.410$$

$$t=30: |dh| = 0.021 \text{ (20x smaller per 10 steps)}$$

$$t=20: |dh| = 0.0011$$

$$t=10: |dh| = 0.000058$$

$$t=0: |dh| = 0.000003 \leftarrow 137,000x \text{ smaller than at } t=39$$

The gradient for position 0 is effectively ZERO.

Weights that affect position 0 receive zero gradient signal.

They cannot be updated correctly. They cannot learn long-range patterns.

THE EXPLODING GRADIENT (the other problem):

If $||W_{hh}|| > 1$: gradient grows exponentially -> NaN in ~200 steps.

Fix: gradient clipping (clip global norm to 5.0).

This is MANDATORY for RNNs. Without it: training crashes.

WHY RNN PLATEAUS AT SAME LEVEL AS MLP:

MLP: fixed 8-position window, no long-range info

RNN: larger context (40 chars), but gradient vanishes before step 10

Both see effectively ~8-10 positions of useful context.

Same architectural ceiling: ppl ~6.2-6.4

TRANSFORMER FIX:

Instead of one h_t , keep ALL past tokens accessible.

Attention computes $\text{score}(i,j) = Q_i \cdot K_j$ directly.

'Romeo' at position 0 to 'Juliet' at position 100: one dot product.

Gradient flows through one attention weight, not 100 tanh multiplications.

ASSIGNMENT TASKS

- Task 1: Implement `RNN::step(x:&Tensor, h:&Tensor)->(logits:Tensor,h_new:Tensor)`
- Task 2: Implement BPTT that logs $|dh_t|$ at every 5th step backward
- Task 3: Train on Shakespeare 5000 steps. Print ppl every 500 steps.
- Task 4: Show gradient norm table: t=40,30,20,10,0. Must show exponential decay.
- Task 5: Compare: print MLP plateau ppl, RNN plateau ppl, then say "Transformer needed".
- Task 6: Show that WITHOUT gradient clipping, loss becomes NaN within 500 steps.

RUST STARTER CODE

```
use crate::tensor::Tensor; pub struct RNN{ pub w_xh:Tensor, pub w_hh:Tensor, pub w_hy:Tensor, pub b_h:Tensor, pub
b_y:Tensor, } impl RNN{ pub fn step(&self,x:&Tensor,h:&Tensor)->(Tensor,Tensor){ let hid=self.b_h.data.len(); let
voc=self.b_y.data.len(); let mut h_new=self.w_xh.matmul(&x.reshape(vec![voc,1])).reshape(vec![hid]).elem_add(
&self.w_hh.matmul(&h.reshape(vec![hid,1])).reshape(vec![hid])).elem_add(&self.b_h); h_new=h_new.apply(|v|v.tanh());
let lo=self.w_hy.matmul(&h_new.reshape(vec![hid,1])).reshape(vec![voc]).elem_add(&self.b_y); (lo,h_new) } pub fn
bptt_log_norms(&self,inputs:&[usize],_targets:&[usize]){ let hid=self.b_h.data.len(); let voc=self.b_y.data.len();
let mut hs=vec![Tensor::zeros(vec![hid])]; for &inp in inputs{ let mut x=Tensor::zeros(vec![voc]); x.data[inp]=1.0;
let (_,hn)=self.step(&x,hs.last().unwrap()); hs.push(hn); } // Backward: track |dh_t| at each step let mut
dh=Tensor::zeros(vec![hid]); dh.data[0]=1.0; for t in (0..inputs.len()).rev(){ let
dh_raw=Tensor::new(dh.data.iter().zip(&hs[t+1].data).map(|(&d,&h)|d*(1.-h*h)).collect(),vec![hid]);
dh=self.w_hh.transpose_2d().matmul(&dh_raw.reshape(vec![hid,1])).reshape(vec![hid]); if t%5==0{ let
norm:f32=dh.data.iter().map(|&v|v*v).sum:::<f32>().sqrt(); eprintln!("t={:>3} |dh|={:.2e}",t,norm); } } }
```

STRETCH GOALS

1. Implement LSTM cell: forget gate, input gate, cell state, output gate.
Show LSTM plateau ppl ~5.8 vs vanilla RNN ~6.4. Gates help but don't fully solve it.
2. Plot the gradient norm decay as ASCII. Show exponential curve.
3. Try T=10 (short), T=40 (medium), T=100 (long). Show plateau vs T.

RESOURCES

- * Karpathy char-rnn (original implementation)
- * The Unreasonable Effectiveness of RNNs (Karpathy blog)
- * Colah — Understanding LSTMs

WHAT YOU WILL FEEL WHEN IT WORKS:

t=39: 0.41. t=0: 0.000003. You are reading the mathematical reason the entire AI field replaced RNNs in 2017.

Phase 4 — Transformer Architecture

A12 Attention — Trace 'To Be or Not to Be' Token by Token

Duration:
1 week

Dataset: 'to be or not to be' — 6 tokens, trace full computation, print attention matrix

Why: Attention must be understood before implemented. Trace the full 6x6 computation on real tokens, every Tensor shape labeled, so the mechanism is arithmetic rather than magic.

THEORY & CONCEPT

SCALED DOT-PRODUCT ATTENTION:

$\text{Attention}(Q, K, V) = \text{softmax}(Q @ K^T / \sqrt{d_k}) @ V$

Q [T,d_k]: queries — "what am I looking for?"

K [T,d_k]: keys — "what do I contain?"

V [T,d_v]: values — "what do I return if selected?"

STEP BY STEP for T=6 tokens, d_k=4:

1. $\text{scores} = Q.\text{matmul}(K.\text{transpose_2d}()).\text{scale}(1/\sqrt{d_k})$ [6,6]

$\text{scores}[i,j] = \text{dot}(Q_i, K_j) / \sqrt{4}$

= "how much should token i attend to token j?"

2. Causal mask (decoder-only / autoregressive):

$\text{scores}[i,j] = -\text{infinity}$ for $j > i$

Token i cannot see future tokens i+1, i+2...

After softmax: attention to future tokens = 0

3. $\text{weights} = \text{softmax_rows}(\text{scores})$ [6,6]

Each row sums to 1. $\text{weights}[i,j]$ = attention from i to j.

4. $\text{output} = \text{weights}.\text{matmul}(V)$ [6,d_v]

$\text{output}[i]$ = weighted sum of all value vectors

= "what information does token i collect from the sequence?"

ATTENTION WEIGHTS (you will compute these):

to be or not to be

to 1.000 0.000 0.000 0.000 0.000 0.000 <- can only see itself

be 0.380 0.620 0.000 0.000 0.000 0.000 <- looks back at "to" (38%)

or 0.271 0.365 0.364 0.000 0.000 0.000

not 0.261 0.222 0.213 0.304 0.000 0.000

KEY INSIGHT:

"be" (second token) attends 38% to "to" and 62% to itself.

It dynamically found its grammatical antecedent.

An MLP uses FIXED position weights. It cannot do this.

Attention is dynamic routing — computed from content, not position.

WHY SCALE BY $\sqrt{d_k}$?

With large d_k , dot products grow in magnitude proportional to d_k .
Large dot products push softmax into saturation: $\text{softmax}([100, 1]) \rightarrow [-1, -0]$
Gradient of saturated softmax $\approx 0 \rightarrow$ vanishing gradient in attention.
Dividing by $\sqrt{d_k}$ keeps dot products in a reasonable range.

ASSIGNMENT TASKS

- Task 1: Use Tensor ops: `scores = Q.matmul(&K.transpose_2d()).scale(1/sqrt(d_k))`
- Task 2: Apply causal mask: set `scores.data[i*T+j]=-inf` for $j > i$
- Task 3: `weights = scores_masked.softmax_rows()`
- Task 4: `output = weights.matmul(&V)`
- Task 5: Print the attention weight matrix as a formatted grid with token names.
- Task 6: Verify: `weights.data[0*T+1..0*T+T]` are all 0 (first token sees nothing else)
- Verify: each row sums to 1.0 within $1e-5$

RUST STARTER CODE

```
use crate::tensor::Tensor; pub fn attention(q:&Tensor,k:&Tensor,v:&Tensor,causal:bool)->Tensor{
    let(t,dk)=(q.shape[0],q.shape[1]); let scale=(dk as f32).sqrt(); // Step 1: scores = Q @ K^T / sqrt(dk) let mut
    scores=q.matmul(&k.transpose_2d()).scale(1./scale); // Step 2: causal mask if causal{for i in 0..t{for j in
    i+1..t{scores.data[i*t+j]=f32::NEG_INFINITY;}} // Step 3: softmax row-wise let weights=scores.softmax_rows(); //
    Step 4: output = weights @ V let out=weights.matmul(v); // Print matrix let toks=["to","be","or","not","to","be"];
    print!("{:<6}", ""); for tk in &toks[..t]{print!("{:>7}",tk);} println(); for i in 0..t{ print!("{:<6}",toks[i]); for
    j in 0..t{print!("{:>7.3}",weights.data[i*t+j]);} println(); } out }
```

STRETCH GOALS

1. Run same attention WITHOUT causal mask (bidirectional). Print matrix.
Show 'to' (first) now also attends forward to 'be', 'or', 'not'.
This is BERT-style encoding attention.
2. Add attention entropy per token: $H_i = -\sum_j w[i,j] \log(w[i,j])$
Later tokens (more context) should have higher entropy.

RESOURCES

- * Attention Is All You Need (Vaswani et al. 2017)
- * The Illustrated Transformer (Jay Alammar)
- * 3Blue1Brown — Attention in transformers

WHAT YOU WILL FEEL WHEN IT WORKS:

Print the matrix. 'be' attends 38% to 'to'. Computed from $Q \cdot K$ dot products, not hardcoded. The network learned this from data.

A13 Multi-Head Attention + RoPE

Dataset: Shakespeare sentences — 4 heads show different specialization patterns

Duration:
1–2
weeks

Why: Multiple attention heads allow the model to attend for different reasons simultaneously — syntax, semantics, coreference. RoPE encodes position geometrically, generalizing beyond training length. Both are used in Llama 3 and every modern LLM.

THEORY & CONCEPT

MULTI-HEAD ATTENTION:

Instead of one attention computation with d_{model} dimensions,
run H parallel attentions each with d_{model}/H dimensions,
then concatenate results.

Motivation: one head might learn syntactic adjacency, another might
learn semantic similarity. H independent attention patterns.

$Q_h = X @ W_{q_h}$ for $h=1..H$ $[T, \text{head_dim}]$

$K_h = X @ W_{k_h}$ $[T, \text{head_dim}]$

$V_h = X @ W_{v_h}$ $[T, \text{head_dim}]$

$\text{head}_h = \text{attention}(Q_h, K_h, V_h)$ $[T, \text{head_dim}]$

$\text{output} = \text{concat}(\text{head}_1..\text{head}_H) @ W_o$ $[T, d_{\text{model}}]$

HEAD SPECIALIZATION (emerges from training, not designed):

Head 1: tends to attend locally (i to $i-1$) – syntactic

Head 2: attends to sentence start – discourse structure

Head 3: distributes attention broadly – global context

Head 4: attends to semantically similar tokens

This specialization appears WITHOUT explicit training for it.

The gradient descent finds it useful and reinforces it.

ROPE – ROTARY POSITION EMBEDDING (Su et al. 2021):

Used in: Llama 2, Llama 3, Falcon, GPT-NeoX, Mistral.

Instead of adding position to embeddings (absolute PE),

ROTATE the Q and K vectors by an angle proportional to position.

For position pos , dimension pair $(2i, 2i+1)$:

$\text{theta}_i = 1 / 10000^{(2i / \text{head_dim})}$

$q'[2i] = q[2i] * \cos(\text{pos} * \text{theta}_i) - q[2i+1] * \sin(\text{pos} * \text{theta}_i)$

$q'[2i+1] = q[2i] * \sin(\text{pos} * \text{theta}_i) + q[2i+1] * \cos(\text{pos} * \text{theta}_i)$

Property: $\text{dot}(q_{\text{pos}_i}, k_{\text{pos}_j})$ depends only on $(j-i)$, not absolute positions.

-> Model learns relative distances, not absolute positions.

-> Generalizes to longer sequences than seen during training.

Precompute $\cos/\sin$ cache at initialization: $O(\text{max_seq} * \text{head_dim})$

Apply at each forward pass: $O(T * \text{head_dim})$ – very cheap.

WHY NOT LEARNED ABSOLUTE PE (GPT-2 style):

Learned PE has a fixed maximum sequence length.

Beyond training length: extrapolation fails.

RoPE naturally extrapolates to longer sequences.

ASSIGNMENT TASKS

■ Task 1: Build MHA struct: wq, wk, wv, wo Tensors + $\cos_cache, \sin_cache$ Vectors

- Task 2: Implement `apply_rope(x:&mut Tensor, t:usize)` using precomputed cache
- Task 3: Implement `forward(x:&Tensor, t:usize)->Tensor`
- Split into H heads, apply rope to Q and K, attention, concat, project
- Task 4: Print all 4 attention matrices on "the quick brown fox jumps". Show they differ.
- Task 5: Compute entropy per head per token. Verify different heads have different profiles.

RUST STARTER CODE

```
use crate::tensor::Tensor; pub struct MHA{ pub wq:Tensor, pub wk:Tensor, pub wv:Tensor, pub wo:Tensor,
cos_cache:Vec<f32>, sin_cache:Vec<f32>, pub n_heads:usize, pub head_dim:usize, pub d:usize, } impl MHA{ pub fn
new(d:usize, n_heads:usize, max_seq:usize)->Self{ let hd=d/n_heads; let s=0.02f32; let(mut cos, mut
sin)=(vec![0.0f32;max_seq*hd/2], vec![0.0f32;max_seq*hd/2]); for pos in 0..max_seq{for i in 0..hd/2{ let
theta=1./10000f32.powf(2.*i as f32/hd as f32); let a=pos as f32*theta; cos[pos*hd/2+i]=a.cos();
sin[pos*hd/2+i]=a.sin(); }} MHA{wq:Tensor::randn(vec![d,d], s), wk:Tensor::randn(vec![d,d], s),
wv:Tensor::randn(vec![d,d], s), wo:Tensor::randn(vec![d,d], s), cos_cache:cos, sin_cache:sin, n_heads:hd, head_dim:hd, d} } fn
rope(&self, x:&mut Tensor, t:usize){ let hd=self.head_dim; for s in 0..t{for i in 0..hd/2{ let idx=s*hd+2*i;
let(x1, x2)=(x.data[idx], x.data[idx+1]); let(c, sn)=(self.cos_cache[s*hd/2+i], self.sin_cache[s*hd/2+i]);
x.data[idx]=x1*c-x2*sn; x.data[idx+1]=x1*sn+x2*c; }} } pub fn forward(&self, x:&Tensor, t:usize)->Tensor{ let d=self.d;
let mut q=x.matmul(&self.wq); let mut k=x.matmul(&self.wk); let v=x.matmul(&self.wv); self.rope(&mut q, t);
self.rope(&mut k, t); // TODO: split into heads, compute per-head attention, concat, project // This is the
architecture implementation exercise let concat=multi_head_attention(&q, &k, &v, t, self.n_heads, self.head_dim);
concat.matmul(&self.wo) } }
```

STRETCH GOALS

1. Implement Grouped Query Attention (GQA): H query heads but G<H key/value heads.
Each group of H/G query heads shares one K,V head.
Llama 3 uses H=32 Q heads and G=8 KV heads -> 4x KV cache reduction.
2. Profile: how much slower is MHA(H=8) vs single-head attention on T=512?

RESOURCES

- * RoPE paper (Su et al. 2021)
- * The Illustrated GPT-2 (Jay Alammar)
- * Karpathy — Lets build GPT (full 2h)

WHAT YOU WILL FEEL WHEN IT WORKS:

4 matrices, same sentence, all different. Nobody instructed them to specialize. The gradient descent found different patterns useful.

A14

Full Transformer Block — Beat the MLP and RNN

Dataset: tiny_shakespeare.txt — MLP ppl=6.2, RNN ppl=6.4, Transformer ppl=4.1

Duration:

1-2
weeks

Why: This is the proof. Same dataset, same training budget, same evaluation metric. Architecture change -> 34% better perplexity. This is why the entire AI field moved to Transformers.

THEORY & CONCEPT

TRANSFORMER BLOCK (pre-norm, modern style used in Llama):

`x = x + MHA(RMSNorm(x))` <- attention sublayer + residual

`x = x + FFN(RMSNorm(x))` <- feedforward sublayer + residual

RMSNORM — ROOT MEAN SQUARE NORMALIZATION:

```
rms = sqrt(mean(x^2) + epsilon)
out = (x / rms) * gamma (no mean subtraction, unlike LayerNorm)
vs LayerNorm: ~15% faster (no mean computation), same quality.
Used in: Llama, T5, PaLM, most modern LLMs.
```

RESIDUAL CONNECTIONS:

```
x = x + sublayer(x)
```

Why: gradient of $x + \text{sublayer}(x)$ w.r.t. $x = 1 + d\text{Sublayer}/dx$

The '+1' provides a direct gradient path bypassing the sublayer.

Without residuals: gradient must flow through ALL sublayers -> vanishes.

With residuals: gradient can skip layers -> trains 100+ layer networks.

SWIGLU FFN — GATED LINEAR UNIT:

```
gate = SiLU(x @ W_g) [T, d_ff]
```

```
up = x @ W_u [T, d_ff]
```

```
out = (gate * up) @ W_d [T, d]
```

$\text{SiLU}(x) = x * \text{sigmoid}(x)$ (smooth, differentiable version of ReLU)

The gate controls which information from 'up' passes through.

Better perplexity than standard FFN+ReLU (Shazeer 2020).

Used in Llama, PaLM, most modern LLMs.

$d_{\text{ff}} = 4 * d_{\text{model}}$ for standard FFN

$d_{\text{ff}} = (8/3) * d_{\text{model}}$ for SwiGLU (adjusted so param count matches)

PERPLEXITY COMPARISON ON tiny_shakespeare.txt:

Architecture Context After 20K steps After 50K steps

MLP (A10) 8 chars ppl = 6.3 ppl = 6.2 <- ceiling

RNN (G01) 40 chars ppl = 6.1 ppl = 6.4 <- same ceiling

Transformer A14 256 chars ppl = 4.8 ppl = 4.1 <- still improving!

WHY: Transformer attends DIRECTLY to any past position.

No gradient chain of 40 tanh multiplications to reach position 0.

'Romeo' at position 0 and 'love' at position 80: one dot product away.

ASSIGNMENT TASKS

- Task 1: `rms_norm(x:&Tensor, w:&Tensor)->Tensor` using Tensor ops
- Task 2: `silu(x:f32)->f32 = x*sigmoid(x)`. `swiglu(x,wg,wu,wd)->Tensor` using `Tensor.matmul`
- Task 3: `Block::forward(x:&Tensor,t:usize)->Tensor` with residual connections
- Task 4: Stack 2 blocks, verify output shape equals input shape
- Task 5: Run gradient checker on full block (all ~5000 params). All must pass.
- Task 6: Train on Shakespeare 50K steps. Print ppl every 5K. Compare to MLP table.

RUST STARTER CODE

```
use crate::tensor::Tensor; pub fn rms_norm(x:&Tensor,w:&Tensor)->Tensor{ let
rms=(x.data.iter().map(|v|v*v).sum:<f32>()/x.data.len() as f32+1e-5).sqrt();
Tensor::new(x.data.iter().zip(&w.data).map(|(xi,wi)|xi/rms*wi).collect(), x.shape.clone()) } fn
silu(x:f32)->f32{x/(1.+(- x).exp())} pub fn swiglu(x:&Tensor,wg:&Tensor,wu:&Tensor,wd:&Tensor)->Tensor{ let
gate=x.matmul(wg).apply(silu); let up=x.matmul(wu); gate.elem_mul(&up).matmul(wd) } pub struct Block{ pub
ln1:Tensor, pub ln2:Tensor, pub attn:MHA, pub wg:Tensor, pub wu:Tensor, pub wd:Tensor, } impl Block{ pub fn
forward(&self,x:&Tensor,t:usize)->Tensor{ let d=self.ln1.data.len(); // Pre-norm attention + residual let
normed1:Vec<f32>=x.data.chunks(d).flat_map(|tok|{
rms_norm(&Tensor::new(tok.to_vec(),vec![d]),&self.ln1.data)}.collect(); let
a_out=self.attn.forward(&Tensor::new(normed1,x.shape.clone()),t); let x=x.elem_add(&a_out); // Pre-norm FFN +
residual let normed2:Vec<f32>=x.data.chunks(d).flat_map(|tok|{
rms_norm(&Tensor::new(tok.to_vec(),vec![d]),&self.ln2.data)}.collect(); let
f_out=swiglu(&Tensor::new(normed2,x.shape.clone()),&self.wg,&self.wu,&self.wd); x.elem_add(&f_out) } }
```

STRETCH GOALS

1. Run gradient checker on full block. Measure: how many of 5000 params pass?
2. Try 1 vs 2 vs 4 blocks. Plot ppl vs num_layers.
3. Implement pre-norm (modern) vs post-norm (original Transformer). Compare training stability.

RESOURCES

- * Karpathy — Lets build GPT (complete)
- * Attention Is All You Need — read in full
- * RMSNorm paper (Zhang and Sennrich)
- * SwiGLU paper (Shazeer 2020)

WHAT YOU WILL FEEL WHEN IT WORKS:

MLP=6.2. RNN=6.4. Transformer=4.1. Same Shakespeare. 34% better. Architecture made the difference.

Phase 5 — Tokenizer

A15 BPE Tokenizer — Train on Shakespeare, Measure Compression

Duration:
1 week

Dataset: tiny_shakespeare.txt — train vocab=512, measure 2.70x compression ratio

Why: The model never sees text. It sees token IDs. BPE is the algorithm that creates this mapping — and for a coding model it must be retrained on code to get good compression ratios on Rust source.

THEORY & CONCEPT

BYTE PAIR ENCODING (BPE) — Sennrich et al. 2015:

Original use: neural machine translation subword units.

GPT-2 adaptation: byte-level BPE (handles all Unicode via UTF-8 bytes).

ALGORITHM:

1. Initialize vocab with 256 byte tokens (0-255)
2. Encode corpus as flat list of byte token IDs
3. Count frequencies of all adjacent token pairs
4. Find most frequent pair (a, b)
5. Create new token ab, add to vocab
6. Replace all (a,b) occurrences in corpus with new token ID
7. Repeat from step 3 until vocab_size reached

WHY START FROM BYTES:

UTF-8 encodes all of Unicode in 1-4 bytes.

Starting from raw bytes: handles ALL text (English, Chinese, emoji, Rust code).

No out-of-vocabulary words — any byte sequence can be represented.

GPT-4 starts from bytes. Llama 3 starts from bytes.

COMPRESSION TRAINING LOG:

Round 1: merge ('e', ' ') -> tok256 freq=28391 ("e_" very common)

Round 2: merge ('t', 'h') -> tok257 freq=22140 ("th" in "the", "that", "this")

Round 3: merge (257, 'e') -> tok258 freq=21883 ("the" = single token!)

Round 50: "and " -> single token

Round512: common 2-3 word phrases -> single tokens

COMPRESSION RESULTS (tiny_shakespeare.txt):

Original: 1,115,394 bytes

After vocab=512 BPE: 412,847 tokens -> 2.70x compression

"To be or not to be":

Byte-level (A10 MLP): 18 tokens (1 per char)

BPE-512: 7 tokens -> 2.6x more context per attention step!

TOKEN FERTILITY:

fertility = tokens / words

Lower = better compression for that domain

English text BPE on English: ~1.3 tokens/word

English text BPE on Rust code: ~2.8 tokens/word (bad for coding models!)

Rust-trained BPE on Rust code: ~1.5 tokens/word (better)

ASSIGNMENT TASKS

- Task 1: `BPE::train(corpus:&[u8], vocab_size:usize)->BPE`
- Print merge log: round, pair merged, frequency, new vocab size
- Task 2: `BPE::encode(text:&[u8])->Vec<u32>` (apply merges in order)
- Task 3: `BPE::decode(ids:&[u32])->Vec<u8>`
- Task 4: Verify round-trip: `decode(encode(text)) == text` for any valid UTF-8
- Task 5: Measure compression: print ratio and total token count
- Task 6: Compare: encode "To be or not to be" (18 chars -> how many tokens?)

RUST STARTER CODE

```
pub struct BPE{ pub merges:Vec<(u32,u32)>, pub merge_rank:std::collections::HashMap<(u32,u32),u32>, pub
vocab:Vec<Vec<u8>>, } impl BPE{ pub fn train(corpus:&[u8],vocab_size:usize)->Self{ let mut
vocab:Vec<Vec<u8>>=(0u8..=255).map(|b|vec![b]).collect(); let mut tokens:Vec<u32>=corpus.iter().map(|&b|b as
u32).collect(); let mut merges=Vec::new(); let mut rank:std::collections::HashMap<(u32,u32),u32>=Default::default();
while vocab.len()<vocab_size{ let mut freq:std::collections::HashMap<(u32,u32),u32>=Default::default();
tokens.windows(2).for_each(|w|*freq.entry((w[0],w[1])).or_insert(0)+=1); let
Some(&best)=freq.iter().max_by_key(|(&v)|v).map(|(k,_)|k) else{break}; let new_id=vocab.len() as u32; let
nb:Vec<u8>=vocab[best.0 as usize].iter().chain(&vocab[best.1 as usize]).cloned().collect(); if vocab.len()%64==0{ let
s:String=nb.iter().filter(|&b|b.is_ascii_graphic()||b==' ' ).map(|&b|b as char).collect(); eprintln!("vocab={:>4}
merging:'{' freq={}",vocab.len(),s,freq[&best]); } rank.insert(best,merges.len() as u32); merges.push(best);
vocab.push(nb); let mut nt=Vec::with_capacity(tokens.len()); let mut i=0; while i<tokens.len(){ if
i+1<tokens.len()&&tokens[i]==best.0&&tokens[i+1]==best.1{nt.push(new_id);i+=2;} else{nt.push(tokens[i]);i+=1;} }
tokens=nt; } println!("Compression: {:.2}x ({} tokens from {} bytes)", corpus.len() as f32/tokens.len() as
f32,tokens.len(),corpus.len()); BPE{merges,merge_rank:rank,vocab} } }
```

STRETCH GOALS

1. Train BPE on Rust source code. Measure fertility on Rust vs English text.

Target: Rust BPE should have ~1.5 tokens/word on Rust code.

2. Add special tokens: `<|bos|>=256+merges`, `<|eos|>=257+merges`.

These are used as sequence delimiters in training and generation.

RESOURCES

- * Karpathy — Lets build the GPT Tokenizer (1h)
- * BPE paper (Sennrich et al. 2015)
- * tiktoken GitHub — GPT-4 tokenizer

WHAT YOU WILL FEEL WHEN IT WORKS:

Watch 'the' become one token in 3 merges. 2.70x compression from a greedy loop. Now the model sees 2.6x more context per attention step.

Gap 2 — Trie Tokenizer

G02 Trie Vocabulary — $O(N*L)$ Encoding, 150x Faster

Duration:
4–6 days

Dataset: tiny_shakespeare.txt — benchmark: naive $O(N*M)$ vs Trie $O(N*L)$

*Why: DeepSeek explicitly listed this assignment. Naive BPE encoding is $O(N*M)$: for 700K lines of Rust source code it takes 20 minutes per encoding pass. Trie reduces it to 8 seconds. Without this, the coding model (A19) is not trainable in finite time.*

THEORY & CONCEPT

NAIVE BPE ENCODING — $O(N * M)$:

For each position i in the token list:

Loop over all M merge rules in order:

If $\text{tokens}[i] == \text{rule.a}$ and $\text{tokens}[i+1] == \text{rule.b}$: apply merge

$N = 2M$ tokens in Shakespeare corpus

$M = 8192$ merge rules

Total: $2M * 8192 = 16$ BILLION comparisons -> ~20 minutes

TRIE — PREFIX TREE DATA STRUCTURE:

Each path from root to a leaf spells out a valid token.

Interior nodes may also be valid tokens.

root

/ \

t e

/ \

th e_

/ \

the th_

|

the_

At each position in text:

Walk down the trie following bytes.

Deepest reachable node with a `token_id` = longest matching token.

Advance position by length of that match.

LONGEST-MATCH ENCODING — $O(N * L)$:

N = number of positions = $2M$ (corpus size)

L = max token length ~ 20 chars (deepest trie path)

Total: $2M * 20 = 40M$ comparisons -> ~0.1 seconds (200x faster)

TRIE NODE STRUCTURE:

```
struct TrieNode {  
    children: HashMap<u8, Box<TrieNode>>, // byte -> child node
```

```
token_id: Option<u32>, // Some(id) if this ends a token
}

Build: for each token id, walk its byte sequence,
create nodes as needed, set token_id at the end.
Query: walk following bytes until no child matches,
track last seen token_id and position.
```

BENCHMARK RESULTS:

```
tiny_shakespeare.txt (1.1MB):

Naive encode: 45 seconds

Trie encode: 0.3 seconds (150x faster)

Output: identical (verified by assert_eq!)

Rust corpus 28MB:

Naive: ~20 minutes

Trie: ~8 seconds (the difference between trainable and not)
```

ASSIGNMENT TASKS

- Task 1: Implement `TrieNode { children:HashMap<u8,Box<TrieNode>>, token_id:Option<u32> }`
- Task 2: `VocabTrie::build(vocab:Vec<Vec<u8>>)->VocabTrie` (insert all tokens)
- Task 3: `VocabTrie::longest_match(text:&[u8], pos:usize)->(token_id:u32, len:usize)`
- Task 4: `VocabTrie::encode(text:&[u8])->Vec<u32>` (loop calling `longest_match`)
- Task 5: Verify: `trie.encode(text) == bpe.encode_naive(text)` with `assert_eq!`
- Task 6: Benchmark both on `tiny_shakespeare.txt`. Print speedup ratio.

RUST STARTER CODE

```
#[derive(Default)] pub struct TrieNode{ pub children:std::collections::HashMap<u8,Box<TrieNode>>, pub
token_id:Option<u32>, } pub struct VocabTrie{root:TrieNode,pub vocab:Vec<Vec<u8>> } impl VocabTrie{ pub fn
build(vocab:Vec<Vec<u8>>)->Self{ let mut root=TrieNode::default(); for(id,bytes)in vocab.iter().enumerate(){ let mut
n=&mut root; for&b in bytes{ n=n.children.entry(b).or_insert_with(||Box::new(TrieNode::default())); }
n.token_id=Some(id as u32); } VocabTrie{root,vocab} } pub fn longest_match(&self,text:&[u8],pos:usize)->(u32,usize){
let(mut best_id,mut best_len)=(text[pos] as u32,1usize); let mut node=&self.root; let mut len=0; while
pos+len<text.len(){ match node.children.get(&text[pos+len]){ None=>break, Some(child)=>{len+=1;node=child; if let
Some(id)=node.token_id{best_id=id;best_len=len;}} } } (best_id,best_len) } pub fn encode(&self,text:&[u8])->Vec<u32>{
let mut ids=Vec::with_capacity(text.len()/3); let mut pos=0; while pos<text.len(){
let(id,len)=self.longest_match(text,pos); ids.push(id); pos+=len; } ids } pub fn decode(&self,ids:&[u32])->Vec<u8>{
ids.iter().flat_map(|&id|self.vocab[id as usize].iter().cloned()).collect() } }
```

STRETCH GOALS

1. Add parallel encoding with Rayon: split corpus at newlines, encode chunks on separate threads.
Measure speedup vs core count. Target: near-linear scaling to 4 cores.
2. Measure memory: how large is the Trie for vocab=512? For vocab=8192?
Compare to `HashMap<Vec<u8>,u32>`. Show the memory/speed tradeoff.

RESOURCES

- * Wikipedia — Trie data structure
- * Aho-Corasick algorithm (production multi-pattern matching)

* memmap2 crate — zero-copy file loading

WHAT YOU WILL FEEL WHEN IT WORKS:

45 seconds -> 0.3 seconds. Same output, verified. At 28MB Rust corpus: naive=20min, Trie=8sec. That is the difference between a trainable model and a planned-but-never-run model.

Phase 6 — Systems & Speed

A16 SIMD GEMM — Make Tensor.matmul 18x Faster

Duration:
1–2
weeks

Dataset: Tensor([512,512]) @ Tensor([512,512]) — benchmark 4 implementations

Why: Tensor.matmul from A02 is correct but ~800ms for [512,512]. GEMM is 90% of LLM compute. You need to understand why it is slow and how to fix it — each fix is one idea, cleanly measurable. Final target: 45ms (18x faster).

THEORY & CONCEPT

WHY NAIVE MATMUL IS SLOW — LOOP ORDER:

```
Naive (i,j,k):
for i in 0..M:
  for j in 0..N:
    for k in 0..K: C[i,j] += A[i,k] * B[k,j]

B[k,j] for fixed j, varying k: stride=N floats=N*4 bytes
For N=512: stride=2048 bytes. Cache line=64 bytes=16 floats.
Each B access loads 1 useful float, evicts 15 others. WASTE.

Reordered (i,k,j) – simple fix, same output:
for i in 0..M:
  for k in 0..K:
    a_val = A[i,k] <- load ONCE outside inner loop
    for j in 0..N: C[i,j] += a_val * B[k,j]

B[k,j..j+N] for fixed k: sequential! Cache line holds 16 floats.
Each B load uses all 16 elements in the cache line. 4x speedup.

BLOCKED (TILED) MATMUL:
Divide into BLOCK x BLOCK tiles that fit in L1 cache (32KB).
For f32: BLOCK = sqrt(32768 / 3 / 4) ~ 52. Use 64.

Within each tile: A_tile, B_tile, C_tile all stay in cache.
Zero cache misses during tile computation. ~2x more speedup.
```

AVX2 SIMD — 8 FLOATS IN ONE INSTRUCTION:

```
__m256 = register holding 8 x f32 simultaneously
_mm256_fmadd_ps(a, b, c): fused multiply-add on 8 floats at once

Instead of: for j in 0..8: c[j] += a_val * b[j] (8 operations)
Use: c_vec = _mm256_fmadd_ps(a_broadcast, b_vec, c_vec) (1 operation)

8x arithmetic throughput per CPU cycle.

MEASURED SPEEDUPS (MacBook Pro M2 equivalent):
Naive (i,j,k): ~800ms 0.33 GFLOPS
Reordered (i,k,j): ~180ms 1.50 GFLOPS (4.4x)
Blocked 64x64: ~95ms 2.84 GFLOPS (2x more)
```

AVX2 SIMD: ~45ms 5.71 GFLOPS (2x more)

Total: 18x speedup from 3 independent ideas.

Still 30x from theoretical peak. That gap = why GPUs exist.

REPLACE `Tensor.matmul` WITH `matmul_fast`:

External API unchanged: `Tensor.matmul(&other) -> Tensor`

Internal implementation: AVX2 if available, blocked otherwise.

All callers (attention, FFN, etc.) automatically get the speedup.

ASSIGNMENT TASKS

- Task 1: v1 naive (i,j,k). Benchmark [512,512]@[512,512].
- Task 2: v2 reordered (i,k,j). Same output, verify with `assert_eq!`. Benchmark.
- Task 3: v3 blocked 64x64. Benchmark. Must be faster than v2.
- Task 4: v4 AVX2 SIMD (x86_64 only). Use `#[target_feature(enable="avx2,fma")]`.
- Scalar fallback for other architectures.
- Task 5: Replace `Tensor.matmul()` internals with `matmul_fast()`. All A02 tests still pass.
- Task 6: Print speedup table: impl | time | GFLOPS | speedup vs v1

RUST STARTER CODE

```
impl Tensor{ pub fn matmul_fast(&self,o:&Tensor)->Tensor{ let(m,k,n)=(self.shape[0],self.shape[1],o.shape[1]);
assert_eq!(k,o.shape[0],"matmul_fast: {:?}@{:?}",self.shape,o.shape); let mut out=Tensor::zeros(vec![m,n]);
#[cfg(target_arch="x86_64")] if is_x86_feature_detected!("avx2"){ unsafe{matmul_avx2(&self.data,&o.data,&mut
out.data,m,k,n);} return out; } matmul_blocked(&self.data,&o.data,&mut out.data,m,k,n); out }
#[cfg(target_arch="x86_64")] #[target_feature(enable="avx2,fma")] unsafe fn
matmul_avx2(a:&[f32],b:&[f32],c:&mut[f32],m:usize,k:usize,n:usize){ use std::arch::x86_64::*; for i in 0..m{for p in
0..k{ let av=_mm256_set1_ps(a[i*k+p]); let mut j=0; while j+8<=n{ let bv=_mm256_loadu_ps(b.as_ptr().add(p*n+j)); let
cv=_mm256_loadu_ps(c.as_ptr().add(i*n+j)); _mm256_storeu_ps(c.as_mut_ptr().add(i*n+j),_mm256_fmadd_ps(av,bv,cv));
j+=8; } while j<n{c[i*n+j]+=a[i*k+p]*b[p*n+j];j+=1;} }} } fn
matmul_blocked(a:&[f32],b:&[f32],c:&mut[f32],m:usize,k:usize,n:usize){ const B:usize=64; for ii
in(0..m).step_by(B){for kk in(0..k).step_by(B){for jj in(0..n).step_by(B){ for i in ii..(ii+B).min(m){for p in
kk..(kk+B).min(k){let av=a[i*k+p]; for j in jj..(jj+B).min(n){c[i*n+j]+=av*b[p*n+j];}}} }}} }
```

STRETCH GOALS

1. Benchmark shapes: [64,64], [128,128], [256,256], [512,512], [1024,1024].

Plot speedup vs matrix size. AVX2 advantage should grow with size.

2. Implement Flash Attention: compute attention in 64x64 tiles, never materializing [T,T].

For T=1024: standard needs 1024*1024*4=4MB; Flash needs O(1) extra memory.

RESOURCES

- * How to Optimize GEMM (Simon Boehm)
- * Intel Intrinsics Guide — AVX2 reference
- * simdееz crate — portable SIMD
- * criterion — benchmarking crate

WHAT YOU WILL FEEL WHEN IT WORKS:

Naive=800ms. AVX2=45ms. Four versions, four ideas, 18x total. The remaining 30x gap to peak is why GPUs exist.

Gap 3 — Parallel Pipeline

G03 Parallel Data Pipeline — From 43% to 97% Compute Utilization

Duration:
4–6 days

Dataset: TinyStories (~500MB) — measure tokens/sec before and after pipeline

Why: All three AI responses (GPAI, ChatGPT, DeepSeek) mentioned parallel data loading. Without it, the training loop in A17 sits idle ~40% of the time waiting for disk I/O. One `mpsc::sync_channel` call fixes 60 minutes of wasted training time.

THEORY & CONCEPT

THE DATA STARVATION PROBLEM:

Without pipeline, training loop:

```
[load from disk 38ms] [tokenize 10ms] [forward+backward 29ms] repeat
```

CPU utilization: $29 / (38 + 10 + 29) = 38\%$ <- mostly waiting for disk

The GPU/CPU is idle while data is loading. Waste.

PRODUCER-CONSUMER WITH BOUNDED CHANNEL:

Two threads:

LOADER (producer): reads corpus, tokenizes, sends batches to channel

TRAINER (consumer): receives batch, runs forward+backward, updates weights

LOADER stays 1-2 batches ahead. When trainer calls `recv()`, batch is ready.

Timeline with pipeline:

```
LOADER: [load 38ms][load 38ms][load 38ms]...
```

```
TRAINER: [fwd+bwd 29ms][fwd+bwd 29ms]...
```

Utilization: $29 / 30 = 97\%$ <- almost no idle time

Rust: `mpsc::sync_channel(capacity)`

`sync_channel(4) = bounded`: loader blocks when 4 batches are ready.

This prevents the loader from consuming unbounded RAM.

Backpressure: if trainer is slow, loader automatically slows too.

Memory cost: 4 batches * 32 seqs * 256 tokens * 4 bytes = 1MB. Negligible.

GRADIENT ACCUMULATION — SIMULATE LARGER BATCH:

Effective batch = `micro_batch_size * accumulation_steps`

Without accumulation: batch=32, 1 update per batch

With accumulation: batch=32, 8 micro-batches per update

Effective batch = $32 * 8 = 256$ — better gradient estimates

Memory: still 32 sequences at a time (fits in RAM)

Implementation:

```
accumulated_grads = zeros_like_params
```

```
for step in 0..accum_steps:
```

```
  batch = rx.recv()
```

```

loss, grads = forward_backward(batch)

accumulated_grads += grads / accum_steps

optimizer.update(accumulated_grads)

BENCHMARK RESULTS ON TINYSTORIES (100K steps):

Sequential: 117 minutes 12,400 tokens/sec

Parallel (2T): 57 minutes 24,800 tokens/sec

2x wall-clock improvement from 20 lines of threading code.

Download dataset: huggingface.co/datasets/roneneldan/TinyStories

```

ASSIGNMENT TASKS

- Task 1: `spawn_loader(corpus:Vec<u32>,batch_size,seq_len,buffer)->Receiver<Vec<Tensor>>`
- Pre-tokenize entire corpus. Sample random start positions.
- Task 2: Training loop: `rx.recv()` to get batch, `forward+backward`, `optimizer.update()`
- Task 3: Add timing: measure `data_wait_ms` and `compute_ms` per step
- Print every 100 steps: "data_wait: Xms compute: Xms"
- Task 4: Before pipeline: `data_wait > compute`. After: `data_wait < 2ms`.
- Task 5: Implement gradient accumulation for `accum_steps=8`
- Task 6: Run 1000 steps with and without pipeline. Compare tokens/sec.

RUST STARTER CODE

```

use std::sync::mpsc; use crate::tensor::Tensor; pub type Batch=Vec<Tensor>; pub fn
spawn_loader(corpus:Vec<u32>,bs:usize,sl:usize,buf:usize) ->(mpsc::Receiver<Batch>,std::thread::JoinHandle<()>) {
    let(tx,rx)=mpsc::sync_channel(buf); let h=std::thread::spawn(move||{ let n=corpus.len(); let mut seed=42u64; loop{
        let mut batch=Vec::with_capacity(bs); for _ in 0..bs{
            seed=seed.wrapping_mul(6364136223846793005).wrapping_add(1442695040888963407); let start=(seed>>33)as usize%(n-sl-1);
            let seq:Vec<f32>=corpus[start..start+sl+1].iter().map(|&t|t as f32).collect();
            batch.push(Tensor::new(seq,vec![sl+1])); } if tx.send(batch).is_err(){break;} } }); (rx,h) } // Training loop: // let
(rx, _h) = spawn_loader(tokens, 32, 256, 4); // let t_data = std::time::Instant::now(); // let batch =
rx.recv().unwrap(); // let data_ms = t_data.elapsed().as_millis(); // let t_comp = std::time::Instant::now(); // let
(loss, grads) = model.forward_backward(&batch); // let comp_ms = t_comp.elapsed().as_millis(); // if step%100==0 {
println!("data:{data_ms}ms compute:{comp_ms}ms"); }

```

STRETCH GOALS

1. Add validation pipeline: separate channel, compute `val_loss` every 1000 steps.
Use `try_recv()` (non-blocking) so training never stalls for validation.
2. Add a throughput monitor: print tokens/sec rolling average every 100 steps.
3. Try Rayon for parallel tokenization within the loader thread.
Does parallelizing tokenization of 32 sequences improve throughput?

RESOURCES

- * Rust Book ch.16 — `mpsc` channels
- * `std::sync::mpsc` docs — `sync_channel` vs `channel`
- * Karpathy nanoGPT DataLoader design
- * `memmap2` crate

WHAT YOU WILL FEEL WHEN IT WORKS:

data_wait: 38ms -> 1ms. compute: 29ms -> 29ms. Nothing changed but the thread structure. 60 minutes returned.

Phase 7 — Complete LLM

A17 Tiny GPT — 10M Parameters on TinyStories

Duration:
4–6
weeks

Dataset: roneneldan/TinyStories (HuggingFace ~500MB, 2.1M short children's stories)

Why: Integration. Every Tensor op from A02, every backward from A07, every attention head from A13, every BPE token from A15, every SIMD multiply from A16, every parallel batch from G03 — all working simultaneously to produce coherent English stories.

THEORY & CONCEPT

```
CONFIG (10M parameters):  
  
vocab_size=4096 d_model=256 n_heads=8  
  
n_layers=6 d_ff=1024 block_size=256 batch_size=32  
  
All weights stored as Tensor structs.
```

PARAMETER COUNT BREAKDOWN:

```
Token embeddings: Tensor[4096, 256] = 1,048,576  
  
Per transformer block:  
  
Attention Wq,Wk,Wv,Wo: 4 * Tensor[256,256] = 262,144  
  
FFN Wg,Wu,Wd: Tensor[256,1024] + Tensor[256,1024] + Tensor[1024,256] = 786,432  
  
RMSNorm ln1,ln2: 2 * Tensor[256] = 512  
  
Total per block: 1,048,576+512 ~ 1,049,088  
  
6 blocks: 6,294,528  
  
Final RMSNorm: 256  
  
Total: ~10.5M parameters  
  
Memory at float32: 10.5M * 4 bytes = 42MB
```

WEIGHT TYING:

```
lm_head (output projection to vocab) = embed.transpose_2d()  
  
Same Tensor used twice: as input embedding AND output projection.  
  
Saves 4MB (1M parameters). Quality maintained or improved.  
  
Used in GPT-2, Llama, and most transformer LMs.
```

TRAINING MILESTONES:

```
step 1K: ppl=67 "Lily was a little girl. She had a"  
step 5K: ppl=22 "Lily had a dog named Max."  
step 25K: ppl=10 "Once upon a time there was a bunny."  
step 50K: ppl=7.4 "Sara found a box in the attic. Inside..."  
step100K: ppl=6.1 Full coherent stories with beginning, middle, end.
```

CHECKPOINT EVERY 5K STEPS:

```
Save: all Tensor.data as flat Vec<f32> -> binary file  
  
Load: read back, reconstruct Tensor with saved shapes
```

Size: 10M * 4 bytes = 40MB per checkpoint

Verify: load and confirm loss is unchanged

GENERATION:

Temperature=0.8, top-p=0.9 for story generation

Prompt: "Once upon a time there was a"

Expected: "... little girl named Lily who loved to play in the garden."

Dataset download: huggingface.co/datasets/roneneldan/TinyStories

ASSIGNMENT TASKS

- Task 1: GPT struct with embed:Tensor, blocks:Vec<Block>, ln_f:Tensor
- Task 2: GPT::forward(ids:&[u32])>Tensor using all Tensor ops from previous assignments
- Task 3: GPT::save(path) / GPT::load(path) — flat binary serialization
- Task 4: Train 100K steps with AdamW, gradient clipping, cosine LR schedule
- Print loss/ppl every 500 steps. Save checkpoint every 5K steps.
- Task 5: Generate 200-char story from prompt "Once upon a time"
- Task 6: Verify checkpoint: save->load->generate gives same output

RUST STARTER CODE

```
use crate::tensor::Tensor; pub struct GPT{ pub embed:Tensor, // [vocab, d_model] — also used as lm_head pub
blocks:Vec<Block>, pub ln_f:Tensor, // [d_model] final RMSNorm weight pub cfg:Config, } impl GPT{ pub fn
forward(&self,ids:&[u32])>Tensor{ let(t,d)=(ids.len(),self.cfg.d_model); // Embedding lookup using Tensor let
rows:Vec<f32>=ids.iter().flat_map(|&id| self.embed.data[id as usize*d..(id as usize+1)*d].iter().cloned()).collect();
let mut x=Tensor::new(rows,vec![t,d]); for block in &self.blocks{x=block.forward(&x,t);} // Final RMSNorm per token
let normed:Vec<f32>=x.data.chunks(d).flat_map(|tok|{
rms_norm(&Tensor::new(tok.to_vec(),vec![d]),&self.ln_f).data}).collect(); let xn=Tensor::new(normed,vec![t,d]); // LM
head (weight-tied): [t,d] @ [d,vocab] -> [t,vocab] xn.matmul(&self.embed.transpose_2d()) } pub fn
save(&self,path:&str){ let mut all=self.embed.data.clone(); for b in &self.blocks{all.extend(b.all_params());}
all.extend(&self.ln_f.data); let bytes:Vec<u8>=all.iter().flat_map(|&f|f.to_le_bytes()).collect();
std::fs::write(path,&bytes).unwrap(); println!("Saved {} params = {:.1}MB",all.len(),bytes.len() as f32/1e6); } } fn
cosine_lr(step:u64,warmup:u64,total:u64,max:f32,min:f32)->f32{ if step<warmup{return max*step as f32/warmup as f32;}
let p=(step-warmup)as f32/(total-warmup)as f32; min+0.5*(max-min)*(1.+(std::f32::consts::PI*p).cos()) }
```

STRETCH GOALS

1. Profile: where is time spent? print per-layer timing every 1000 steps.
2. Implement learning rate finder: try LR 1e-5 to 1e-2, plot loss vs LR.
3. Try 4 vs 8 attention heads with same total params. Which trains faster?

RESOURCES

- * Build a Large Language Model From Scratch (Raschka)
- * TinyStories dataset (HuggingFace)
- * Karpathy — Lets reproduce GPT-2 (4h)
- * RustGPT GitHub — reference implementation

WHAT YOU WILL FEEL WHEN IT WORKS:

step 100K: 'Once upon a time there was a little girl named Lily.' Every Tensor, every matmul, every attention head, every BPE token — all working. That sentence is the payoff.

A18 KV Cache + Sampling — From 21 to 208 Tokens/Sec

Duration:
1 week

Dataset: Your trained TinyStories GPT — benchmark generation speed

Why: Without KV cache, generating 100 tokens from a 50-token prompt requires $O(T^2)$ attention — each step recomputes K,V for all past tokens. With cache: $O(T)$ per step. Same model, 10x faster. This is the difference between demo and deployable.

THEORY & CONCEPT

WHY GENERATION IS SLOW WITHOUT CACHE:

At step t , to generate token $t+1$:

```
full_context = prompt + generated[0..t] length = 50+t
forward(full_context) -> recomputes K,V for ALL 50+t tokens
```

Only the LAST logit vector is used. All others discarded.

Step 1: `forward(Tensor[50, d]) = 8ms`
Step 50: `forward(Tensor[99, d]) = 31ms`
Step 100: `forward(Tensor[149, d]) = 67ms` (quadratic growth)
Total 100 tokens: ~4.7 seconds -> 21 tokens/sec

KV CACHE — STORE AND REUSE:

During attention: Q, K, V are computed from X.

K and V for past tokens don't change when we add a new token.

PREFILL phase (prompt processing):

```
Process full prompt as Tensor[50, d].
Store K and V from every block in cache.
```

DECODE phase (token generation):

```
New token -> Tensor[1, d]
Compute Q, K, V for this single new token.
Append K, V to cache.
Attention: Q[new] over cached K[0..t] -> O(t) work per step.
Step 1-100: each ~0.4ms. Total: 0.48s -> 208 tokens/sec (10x!)
```

KV CACHE AS TENSOR STORE:

Per layer: `k_cache:Vec<Tensor>, v_cache:Vec<Tensor>`

Each element is a `Tensor[seq_len, head_dim]` for one head.

Append by extending `.data` and updating `.shape`.

Memory: $2 * n_layers * n_heads * head_dim * max_seq * 4 \text{ bytes}$
 $= 2 * 6 * 8 * 32 * 256 * 4 = 6.3\text{MB}$ (negligible)

TOP-P (NUCLEUS) SAMPLING:

Sort tokens by probability descending.

Keep the smallest set of tokens whose cumulative probability $\geq p$.

Sample from this 'nucleus' uniformly.

p=0.9: usually 10-50 tokens in nucleus
p=0.1: usually 1-3 tokens (near-greedy)
p=1.0: sample from full distribution
Industry default: temperature=0.8, top_p=0.9

ASSIGNMENT TASKS

- Task 1: KVCache struct with Vec<(Tensor,Tensor)> per layer, append(), keys(), values()
- Task 2: Modify Block::forward to accept &mut KVCache, append new K,V, attend over full cache
- Task 3: GPT::prefill(ids:&[u32], cache:&mut KVCache)->Tensor (process full prompt)
- Task 4: GPT::decode_step(token_id:u32, cache:&mut KVCache)->Tensor (one new token)
- Task 5: GPT::generate(prompt,max_tokens,temp,top_p)->String using prefill+decode loop
- Task 6: Benchmark: 100 tokens without cache vs with cache. Print tokens/sec for both.

RUST STARTER CODE

```
use crate::tensor::Tensor; pub struct KVCache{ pub kv:Vec<(Tensor,Tensor)>, // (K[seq,d], V[seq,d]) per layer pub seq_len:usize, } impl KVCache{ pub fn new(n_layers:usize,d:usize)->Self{ KVCache{kv:(0..n_layers).map(|_|(Tensor::zeros(vec![0,d]),Tensor::zeros(vec![0,d]))).collect(), seq_len:0} } pub fn append(&mut self, layer:usize, new_k:&Tensor, new_v:&Tensor){ let d=new_k.data.len(); let(k,v)=&mut self.kv[layer]; k.data.extend_from_slice(&new_k.data); v.data.extend_from_slice(&new_v.data); k.shape=vec![k.data.len()/d,d]; k.strides=vec![d,1]; v.shape=vec![v.data.len()/d,d]; v.strides=vec![d,1]; self.seq_len+=1; } } fn top_p_sample(logits:&Tensor, temp:f32, p:f32)->usize{ let s=logits.scale(1./temp); let max=s.max(); let mut probs:Vec<(f32,usize)>=s.data.iter().enumerate().map(|(i,&v)|((v-max).exp(),i)).collect(); let sum:f32=probs.iter().map(|(p,_)|p).sum(); probs.iter_mut().for_each(|(v,_)|*v/=sum); probs.sort_by(|a,b|b.0.partial_cmp(&a.0).unwrap()); let mut cdf=0.; let nuc:Vec<usize>=probs.into_iter().take_while(|(v,_)|{let k=cdf<p;cdf+=v;k}).collect(); let ns:f32=nuc.iter().map(|(v,_)|v).sum(); let r=rand_f32(); let mut c=0.; for(v,i)in&nuc{c+=v/ns;if r<c{return*i;}} nuc[0].1 }
```

STRETCH GOALS

1. Implement beam search (width=5). Compare quality to top-p on same prompt.
2. Profile: which block operation is slowest during decode? Layer attention or FFN?
3. Add streaming output: print each token as it is generated (one char at a time).

RESOURCES

- * vLLM paper — PagedAttention for efficient serving
- * FlashAttention (Dao et al. 2022)
- * Karpathy nanoGPT inference loop

WHAT YOU WILL FEEL WHEN IT WORKS:

21 tok/sec -> 208 tok/sec. Same model, same Tensor weights. The KV cache is the bridge between research and deployment.

Gap 4 — Quantization

G04 INT8 Quantization — Tensor Weights 40MB to 10MB

Duration:
1 week

Dataset: Your trained TinyStories GPT — quantize all weight Tensors, compare ppl

Why: GPT and DeepSeek both listed quantization explicitly. Your GPT weight Tensors are float32. INT8 block quantization gives 4x compression with <0.1 perplexity degradation. This is what makes LLMs run on phones and edge devices.

THEORY & CONCEPT

FLOATING POINT FORMATS:

float32: 32 bits (1 sign + 8 exp + 23 mantissa). Range: $\pm 3.4e38$.

float16: 16 bits (1 sign + 5 exp + 10 mantissa). Range: ± 65504 .

Problem: small range $\rightarrow$ gradients underflow to 0 during training.

Use: inference only.

bfloat16: 16 bits (1 sign + 8 exp + 7 mantissa). Same range as float32.

Use: training on TPUs/GPUs. No range issue. Less precision.

INT8 QUANTIZATION — ABSMAX METHOD:

For a Tensor of weights:

$scale = \max(|weights|) / 127$

$q[i] = \text{clamp}(\text{round}(weights[i] / scale), -128, 127)$ as i8

Storage: 1 byte per weight + 4 bytes for scale = $\sim 4x$ compression.

Dequantize: $w[i] = q[i]$ as f32 * scale

BLOCK QUANTIZATION (better quality):

Instead of one scale per Tensor, use one scale per 32-element block.

Outlier weights in one block don't affect precision in other blocks.

For a [256, 1024] weight Tensor:

$256 * 1024 / 32 = 8192$ blocks

Each block: 32 i8 weights + 1 f32 scale = 36 bytes

Total: $8192 * 36 = 294,912$ bytes vs float32: 1,048,576 bytes = 3.6x

DEQUANTIZE-ON-THE-FLY:

Keep weights as QuantTensor in memory (i8 blocks).

During Tensor.matmul: for each block, dequantize to [f32;32], multiply, accumulate.

Never store the full float32 Tensor in RAM.

Memory bandwidth: reading 1 byte per weight vs 4 bytes.

Arithmetic: still float32 after dequantization.

Net: memory-bound workloads run 2-4x faster.

PERPLEXITY IMPACT (measured on TinyStories):

float32 Tensor: ppl = 6.09 (baseline)

INT8 absmax Tensor: ppl = 6.17 (+0.08 — barely noticeable)

INT8 block-32 Tensor: ppl = 6.11 (+0.02 – essentially identical)

INT4 block-32 Tensor: ppl = 6.58 (+0.49 – some quality loss)

INT8 is the production sweet spot: 4x smaller, indistinguishable quality.

SIZES:

float32 GPT (10M params): 40.0 MB

INT8 QuantTensor: 10.2 MB

INT4: 5.1 MB

Fits on: a phone (INT8), a browser (INT4)

ASSIGNMENT TASKS

- Task 1: QuantBlock { weights:[i8;32], scale:f32 } with quantize/dequantize methods
- Task 2: QuantTensor { blocks:Vec<QuantBlock>, shape:Vec<usize> } from Tensor
- Verify: max quantization error per block < scale/2
- Task 3: matmul_quant(input:&Tensor, weights:&QuantTensor)->Tensor using dequant-on-fly
- Task 4: GPT::quantize(&self)->QuantGPT — convert all weight Tensors to QuantTensors
- Task 5: Measure: float32 ppl, INT8 ppl on TinyStories validation set
- Task 6: Print size comparison: "float32: 40.0MB INT8: 10.2MB reduction: 3.9x"

RUST STARTER CODE

```
use crate::tensor::Tensor; #[repr(C)] pub struct QuantBlock{pub weights:[i8;32],pub scale:f32} impl QuantBlock{ pub
fn quantize(v:&[f32;32])->Self{ let max=v.iter().map(|&x|x.abs()).fold(0.0f32,f32::max); let scale=if
max>0.{max/127.}else{1.}; QuantBlock{weights:std::array::from_fn(|i| (v[i]/scale).round().clamp(-128.,127.))as
i8},scale} } pub fn dequantize(&self)->[f32;32]{self.weights.map(|w|w as f32*self.scale)} pub fn
max_error(&self,orig:&[f32;32])->f32{ let d=self.dequantize();
orig.iter().zip(&d).map(|(a,b)|(a-b).abs()).fold(0.0f32,f32::max) } } pub struct QuantTensor{pub
blocks:Vec<QuantBlock>,pub shape:Vec<usize>} impl QuantTensor{ pub fn from_tensor(t:&Tensor)->Self{
assert_eq!(t.data.len()%32,0,"Tensor length must be divisible by 32"); let blocks=t.data.chunks(32).map(|c|{ let
arr:[f32;32]=c.try_into().unwrap(); QuantBlock::quantize(&arr)}).collect(); QuantTensor{blocks,shape:t.shape.clone()}
} pub fn size_bytes(&self)->usize{self.blocks.len()*36} // 32 i8 + 4 f32 } // Analysis function: pub fn
print_size_comparison(model:&GPT,quant:&QuantGPT){ let f32_mb=model.param_count() as f32*4./1e6; let
int8_mb=quant.size_bytes() as f32/1e6; println!("float32: {:.1}MB INT8: {:.1}MB reduction: {:.1}x",
f32_mb,int8_mb,f32_mb/int8_mb); }
```

STRETCH GOALS

1. Implement INT4: pack 2 weights per byte. fn pack(hi:i8,lo:i8)->u8{((hi&0xF)<<4)|((lo&0xF))}.

Measure: at what point does INT4 quality become unacceptable?

2. Profile which layers are most sensitive to quantization.

Try: keep attention weights float32, quantize FFN only. Measure ppl.

3. Implement GPTQ: instead of absmax, solve $\min ||W - \text{dequant}(\text{quant}(W,s))||^2$ for optimal s.

RESOURCES

- * GPTQ paper (Frantar et al. 2022) — block-wise quantization
- * Tim Dettmers — The case for 4-bit precision
- * half crate — BF16 and F16 in Rust

WHAT YOU WILL FEEL WHEN IT WORKS:

Is -lh: checkpoint.bin=40MB, model_int8.bin=10MB. Generate from both. Indistinguishable. 4x compression from bit operations.

Phase 8 — Rust Coding Model

A19 Rust Code Completion — The Final Project

Duration:
4–8
weeks

Dataset: Rust std library + top crates + your 18 assignments (~700K lines, 28MB)

Why: The loop closes. A model built in Rust, trained on Rust, completing Rust. Every Tensor op from A02, every matmul from A16, every KV Cache Tensor from A18, every QuantTensor from G04 — all participating in generating valid Rust code.

THEORY & CONCEPT

DIFFERENCES FROM TINYSTORIES GPT:

1. TOKENIZER (re-train BPE on Rust source):

Standard BPE on Rust: fertility ~2.8 chars/token (bad)

Rust-trained BPE: fertility ~1.5 chars/token (good)

Improvement: 65% more Rust context per attention window.

Code-specific optimizations in BPE training:

- Weight ' ' (4 spaces) to merge early -> 1 token for indentation
- Let ':' merge early -> 1 token (Rust path separator, very frequent)
- Let '->' merge early -> 1 token (return type arrow)
- These appear in nearly every Rust line, so they get merged in round ~10

2. DATASET STRUCTURE:

File separator token: <|rust|> between files.

This teaches the model file-level context (like Copilot).

Filter: .rs files only, exclude /target/, exclude tests/, tests.rs

Sources:

- Your 17 assignments (~2,500 lines)
- Rust standard library: github.com/rust-lang/rust/library (~200K lines)
- Popular crates source: [serde](#), [tokio](#), [rayon](#), [rand](#) (~500K lines)

Total: ~700K lines = ~28MB raw text

3. EVALUATION SUITE (10 function completions):

```
fn fibonacci(n:u32)->u32{ -> if n<=1{return n;}...
```

```
fn is_prime(n:u64)->bool{ if n<2{return -> false;}
```

```
fn max(a:i32,b:i32)->i32{ if a>b{ -> a}else{b}
```

```
pub struct Stack<T>{ data: -> Vec<T>,
```

```
for i in 0.. -> n{
```

Scoring: 2pts exact + 1pt correct syntax + 1pt compiles = max 40

Target: >=24/40 (60%)

Use temperature=0.3 for evaluation (low, near-greedy for predictable output).

4. ALL TENSOR INFRASTRUCTURE UNCHANGED:

Same Tensor struct from A02.

Same matmul_fast (SIMD) from A16.

Same KVCache holding Tensors from A18.

Same QuantTensor from G04.

Only: different training data and different BPE tokenizer.

WHAT CHANGES BETWEEN STORY MODEL AND CODE MODEL:

Story model: sees "Once upon a time" -> generates fluent English

Code model: sees "fn fibonacci(" -> generates valid Rust

The ONLY difference: training data distribution and tokenizer.

The Transformer architecture, Tensor ops, and training algorithm are identical.

This is the power of the pre-training paradigm.

ASSIGNMENT TASKS

- Task 1: collect_rust(roots:&[&str])->Vec<u8> using walkdir crate. Filter .rs files.
- Task 2: Re-train BPE on Rust corpus (vocab=8192). Measure fertility vs English-trained BPE.
- Task 3: Fine-tune or train from scratch 10M GPT on Rust corpus. Use parallel pipeline (G03).
- Task 4: Implement evaluate(model, tok)->score running all 10 eval cases.
- Task 5: Achieve >=24/40 on evaluation suite.
- Task 6: Quantize to INT8 using G04. Verify QuantGPT passes same evaluation.

RUST STARTER CODE

```
use walkdir::WalkDir; use crate::tensor::Tensor; pub fn collect_rust(roots:&[&str])->Vec<u8>{ let mut
corpus=Vec::new(); for root in roots{ for e in WalkDir::new(root).into_iter().filter_map(|e|e.ok()){ let p=e.path();
if p.extension().map_or(false, |e|e=="rs") &&!p.to_string_lossy().contains("/target/")
&&!p.to_string_lossy().contains("tests/"){ if let Ok(src)=std::fs::read(p){
corpus.extend_from_slice(b"\n<|rust|>\n"); corpus.extend_from_slice(&src); } } } } corpus } struct
Eval{prompt:&'static str,expected:&'static str} const EVALS:&[Eval]=&[ Eval{prompt:"fn fibonacci(n:u32)->u32{\n
",expected:"if n<=1{return n;}", Eval{prompt:"fn is_prime(n:u64)->bool{\n if n<2{return ",expected:"false;}",
Eval{prompt:"fn max(a:i32,b:i32)->i32{\n if a>b{",expected:"a}else{b}", Eval{prompt:"pub struct Stack<T>{\n
data:",expected:"Vec<T>,"}, Eval{prompt:"for i in 0..",expected:"n{", }; pub fn
evaluate(model:&GPT,tok:&BPE)->(u32,u32){ let(mut score,mut max_s)=(0u32,0u32); for e in EVALS{ max_s+=4; let
out=model.generate(tok,e.prompt,50,0.3); let gen=out.trim_start_matches(e.prompt); println!("prompt: {...\ngot:
{}\nwant: {}", &e.prompt[..e.prompt.len().min(40)], &gen[..gen.len().min(50)],e.expected); if
gen.trim_start().starts_with(e.expected){score+=2;println!("[EXACT +2]");} else if
gen.contains(&e.expected[..e.expected.len().min(8)]){score+=1;println!("[PARTIAL +1]");} } println!("Score:
{}/{}",score,max_s); (score,max_s) }
```

STRETCH GOALS

1. Implement a minimal Rust lexer (ident/keyword/op/literal). Use it to measure:
what % of completions produce syntactically valid Rust? What % have balanced braces?
2. Add file-path prefix: "<|file|>src/main.rs\n" before each file in training.
Does the model learn file-specific conventions (e.g. main() structure)?
3. Quantize to INT8 (G04). Run evaluation on QuantGPT. Does score change?

RESOURCES

- * The Stack dataset (HuggingFace) — Rust subset
- * Codex paper (Chen et al. 2021) — evaluation methodology
- * walkdir crate — recursive directory walking

* HumanEval benchmark — study evaluation design

WHAT YOU WILL FEEL WHEN IT WORKS:

*fn fibonacci(n:u32)->u32{\n if n<=1{return n;}\n fibonacci(n-1)+fibonacci(n-2)\n}. Your model completed it. Built in Rust.
Trained on Rust. Completing Rust. The loop is closed.*

After all 23 assignments you will have:

- ✓ Found the golden ratio in Fibonacci using gradient descent (A04)
- ✓ Solved XOR with 9 hand-computed chain rule steps (A06)
- ✓ Classified MNIST digits at >85% accuracy (A09)
- ✓ Watched MLP plateau at ppl=6.2, unable to improve (A10)
- ✓ Watched RNN gradient shrink to 0.000003 at position 0 (G01)
- ✓ Broken through that ceiling with a Transformer at ppl=4.1 (A14)
- ✓ Compressed Shakespeare 2.70x with your own BPE tokenizer (A15)
- ✓ Made matrix multiply 18x faster with AVX2 SIMD (A16)
- ✓ Doubled training throughput with a 20-line parallel pipeline (G03)
- ✓ Trained a 10M parameter GPT on TinyStories to generate stories (A17)
- ✓ Made generation 10x faster with a KV cache (A18)
- ✓ Shrunk the model from 40MB to 10MB with INT8 quantization (G04)
- ✓ Completed Rust functions with a model built in Rust from scratch (A19)

All of this from first principles. No PyTorch. No ML libraries. Every Tensor, every gradient, every attention weight, every token ID computed by code you wrote yourself.